





**DOKUZ EYLUL**
**UNIVERSITY**
**ENGINEERING FACULTY**
**DEPARTMENT OF COMPUTER ENGINEERING**

# CME2210 OBJECT ORIENTED ANALYSIS AND DESIGN

## FINAL REPORT

# Beauty Salon Management System CenGIRL

by
Şengül Akkaya 2023510121
Arzu Hamdioğlu 2023510054
Zeliha Erdoğan 2023510145

**Lecturer**
**Prof. Dr. Semih Utku**
**IZMIR**
**29.05.2026**

# 1. Executive Summary

This document is the final consolidated report for the CME2210 Object Oriented Analysis and Design course project. It unifies the outcomes of Phase 1 (Analysis), Phase 2 (Design), and Phase 3 (Implementation) for the CenGIRL Beauty Salon Management System — a medium-scale Java application that digitizes appointment scheduling, staff calendar management, service catalog administration, notifications, and payment processing for a beauty salon.

The project deliberately prioritizes software design quality over graphical user interfaces or persistent databases. All data is held in in-memory repositories backed by HashMap structures, while the runtime interaction occurs through a structured console menu for three actor types: Customer, Employee, and Manager.

### Architecture Overview

The implementation follows a strict four-layer N-tier architecture:

- Presentation — menus, I/O, formatting
- Application / Service — use-case orchestration, authorization checks
- Domain — entities, value objects, business rules, repository interfaces
- Infrastructure — in-memory persistence, utilities
-

### Key Achievements

- Full requirements-to-code traceability from use cases to service methods
- SOLID-aligned separation of concerns across packages
- Extensible payment and user-creation mechanisms without modifying core domain logic
- Robust exception handling with domain-specific error types
- Documented manual functional test scenarios with expected outcomes
- Core design patterns: Repository, Strategy (payments), Factory Method (user creation), Observer (notifications)

The accompanying Java source code (com.presentation, com.application, com.domain, com.infrastructure) constitutes the executable deliverable referenced throughout Phase 3 of this report.

# 2. Phase 1 — Analysis

## 2.1 System Definition and Project Objectives

### 2.1.1 System Definition

CenGIRL — Beauty Salon Management System is a comprehensive software solution designed to digitize and optimize traditional, manual appointment, staff, and service management processes prevalent in beauty salons. The system addresses three core operational challenges:

- Appointment conflicts and double-booking when multiple customers request the same employee time slot
- Complexity of staff work schedules, including working hours, days off, and per-day availability
- Unstructured payment flows for both customer service fees and internal payroll

**Actor**

- **Customer** : Browse services, view available slots, book/cancel appointments, pay for completed services
- **Employee :** View calendar, update appointment outcomes (completed, no-show, cancelled)
- **Manager :** Manage service catalog and employees, configure schedules, process salaries

All interactions occur through a console-based user interface. Data persists only for the duration of a program run using in-memory data structures, aligning with academic scope constraints.

### 2.1.2 Project Objectives

1. Design-first delivery — Prioritize OOAD quality, layered architecture, and clean code over GUI or database implementation.
2. Java implementation — Implement a medium-scale system in Java using in-memory collections.
3. OOAD and Clean Code — Apply encapsulation, abstraction, polymorphism, and readable naming consistently.
4. N-tier architecture — Enforce Presentation, Application/Service, Domain, and Infrastructure layers with unidirectional dependencies.
5. SOLID and design patterns — Integrate patterns that support extensibility (Strategy, Factory, Observer, Repository).
6. Lifecycle documentation — Produce analysis, design, and implementation artifacts suitable for academic evaluation.

### 2.1.3 Problem Statement

Beauty salons operating without integrated software often rely on paper agendas or informal messaging. This leads to lost revenue from scheduling errors, customer dissatisfaction due to unclear availability, and managerial overhead when tracking payroll and service pricing separately. CenGIRL centralizes these concerns in a single cohesive domain model with explicit state machines for appointments and payments.

### 2.1.4 Scope

**In Scope**

- User registration and authentication (with account lockout)
- Role-based menus for Customer, Employee, Manager
- Service CRUD (Manager)
- Employee onboarding and work schedule configuration (Manager)
- Appointment lifecycle and calendar booking
- Notifications on create/cancel
- Customer payments and manager salary payments

**Out of Scope (by design)**

- Graphical user interface (web/desktop/mobile)
- Persistent database or ORM
- Real payment gateway integration
- SMS/email delivery (notifications are logged to console)

## 2.2 Functional Requirements

### 2.2.1 Customer (Actor) Functions

**Authentication and Profile Management**

- Register a new customer account
- Log in with username/password
- Verify identity; handle wrong login attempts
- View and update profile (name, email, phone)

FR-C-03 detail: After three consecutive failed login attempts (AuthService.LOCK_THRESHOLD = 3), the account is locked until a successful authentication resets counters.

**Appointment and Service Management**

- List all beauty services
- View available time slots for service + employee + date
- Create appointment on available slot
- Trigger Notification Service on successful booking

- View past and current appointments
- Cancel active (Scheduled) appointments
- Release employee calendar slot on cancel
- Trigger Notification Service on cancellation

**Payment Operations**

- Pay for completed appointments
- Select payment method (cash, card, wallet)
- Transition appointment to Paid on success

### 2.2.2 Manager Functions

- Add/update/delete beauty services (name, duration, price)
- Add and manage employee records
- Set employee working hours and days off
- Pay employee salary via Payment Service
- Prevent duplicate salary payment per month

### 2.2.3 Employee Functions

- Log in with employee credentials
- View assigned appointments / calendar
- Update appointment status: Completed, No-Show, Cancelled

### 2.2.4 Core System Services

**Notification Service**

- Responsible for sending status updates to relevant actors (customer and employee) during appointment creation and cancellation.
- Implemented as NotificationService implementing AppointmentObserver; messages are printed to console and stored in an in-memory log.

**Payment Service**

- Centralizes customer appointment payments and manager salary disbursements.
- Uses Strategy pattern for payment method execution.
- Maintains Payment entity state (Pending → Completed/Failed → optional Refunded).

## 2.3 Non-Functional Requirements

### 2.3.1 Maintainability

- Strict layering
- Clean code
- Single responsibility per class

Layer boundaries ensure that UI changes do not affect domain rules, and repository implementations can be replaced without touching Appointment or Payment entities.

### 2.3.2 Extensibility

- Open-Closed: new payment strategies or user roles via extension, not modification of core domain

New payment types are added by implementing PaymentStrategy and registering in PaymentService constructor — no change to Payment.complete() logic.

### 2.3.3 Data Integrity

- No double-booking
- Calendar sync on cancel
- Referential consistency via IDs

EmployeeCalendar.bookSlot() rejects overlapping slots. AppointmentService.cancelAppointment() releases the exact TimeSlot booked at creation time.

### 2.3.4 Security

- Role-based access in services
- Customers cannot cancel others' appointments

Service methods validate actor ownership (customerId, employeeId, managerId) before mutating entities. Presentation layer never bypasses services to touch repositories directly.

### 2.3.5 Performance

- O(1) average lookup in HashMap repositories
- Efficient slot scanning

InMemoryUserRepository, InMemoryAppointmentRepository, etc. use HashMap<String, Entity> for primary-key access, ensuring O(1) average lookup performance.

### 2.3.6 Robustness

- Domain exceptions
- Application does not crash on validation errors

ConsoleApplication catches DomainException and generic Exception, printing user-friendly messages while keeping the main loop alive. Invalid inputs such as wrong date formats or bad menu choices never crash the application.

### 2.3.7 Usability

- Clear console menus, sections, success/error messages

The system provides intuitive console-based navigation by organizing features into logical menus and distinct sections based on user roles (Customer, Employee, Manager). Users receive immediate

feedback through clear success confirmations or descriptive error messages, ensuring a smooth operational flow and easy navigation without needing a graphical interface.

## 2.4 Actors and System Boundary

The system recognizes three human actors and two internal services:

- **Customer (Human Actor):** End user booking beauty services and making payments.
- **Employee (Human Actor):** Service provider managing their personal calendar and updating appointment outcomes.
- **Manager (Human Actor):** Administrative authority over catalog, staff, and payroll.
- **Notification Service (Internal Service):** Observer that dispatches booking/cancellation confirmations.
- **Payment Service (Internal Service):** Centralizes appointment and salary payment processing.

## 2.5 Use Case Specifications

### Use Case UC-01: Create Appointment

| | |
|---|---|
| **Name** | Create Appointment |
| **Primary Actor** | Customer |
| **Included Use Cases** | Notification Service |
| **Preconditions** | Customer authenticated; at least one service and employee exist |
| **Postconditions** | Appointment in Scheduled state; slot booked; notifications sent |

**Main Success Scenario**

7. Customer selects beauty service and employee.
8. System displays available time slots (AppointmentService.getAvailableSlots).
9. Customer selects slot and confirms.
10. System validates availability, persists appointment, books calendar, links appointment to customer.
11. Notification Service notifies customer and employee.

**Extensions**

- 3a. Slot no longer available → ValidationException
- 1a. Invalid service/employee ID → EntityNotFoundException

### Use Case UC-02: Complete Service and Make Payment

| | |
|---|---|
| **Name** | Complete Service and Make Payment |
| **Primary Actors** | Employee, Customer |
| **Included Use Cases** | Payment Service |
| **Preconditions** | Appointment in Scheduled state (for completion) |
| **Postconditions** | Appointment in Paid state; Payment in Completed state |

**Main Success Scenario**

12. Employee marks appointment Completed (AppointmentService.updateStatusByEmployee).
13. Customer initiates payment; system creates/retrieves pending Payment.
14. Customer selects payment strategy (cash, card, digital wallet).
15. PaymentService.processAppointmentPayment executes strategy.
16. On success: payment Completed, appointment moves to Paid status.

### Use Case UC-03: Cancel Appointment

| | |
|---|---|
| **Name** | Cancel Appointment |
| **Primary Actor** | Customer |
| **Included Use Cases** | Notification Service |
| **Preconditions** | Appointment exists in Scheduled state |
| **Postconditions** | Appointment Cancelled; slot freed; both parties notified |

**Main Success Scenario**

17. Customer selects active appointment.
18. Customer confirms cancellation.
19. Status → Cancelled; slot released; repository updated.
20. Notification Service informs both customer and employee.

**Extensions**

- Appointment not in Scheduled state → InvalidStateTransitionException

### Use Case UC-04: Pay Salary

| | |
|---|---|
| **Name** | Pay Salary |
| **Primary Actor** | Manager |
| **Included Use Cases** | Payment Service |

| | |
|---|---|
| **Preconditions** | Employee exists; salary not already paid this month |
| **Postconditions** | Employee salary record marked Paid for current YearMonth |

**Main Success Scenario**

21. Manager selects employee from payroll module.
22. Manager confirms salary amount (from employee record).
23. Payment Service processes via selected strategy.
24. Employee marked salary-paid for current YearMonth.
25. Confirmation message returned to manager.

**Extensions**

- Salary already paid this month → ValidationException (duplicate prevention)

### Use Case UC-05: User Login

**Main Success Scenario**

26. User enters username and password.
27. System locates user, checks lock status, verifies password.
28. On success: failed attempt counter reset; role menu displayed.

**Extensions**

- Wrong password → increment failures; lock at threshold (3)
- Unknown username → AuthenticationException

### Use Case UC-06: Manage Services (Manager)

Manager can add, update, and delete BeautyService entries via ManagerService (name, duration in minutes, price as BigDecimal). All changes are immediately visible to customers browsing the service catalog.

### Use Case UC-07: Manage Employees (Manager)

Manager registers employees with WorkSchedule, monthly salary, and credentials through AuthService.register with role EMPLOYEE. The manager can also update working hours and manage days off via ManagerService.

## 2.6 Use Case Diagram

The Use Case Diagram below illustrates all interactions between system actors (Customer, Employee, Manager) and the core use cases. Internal services (Notification Service, Payment Service) appear as included use cases activated by the primary workflows.

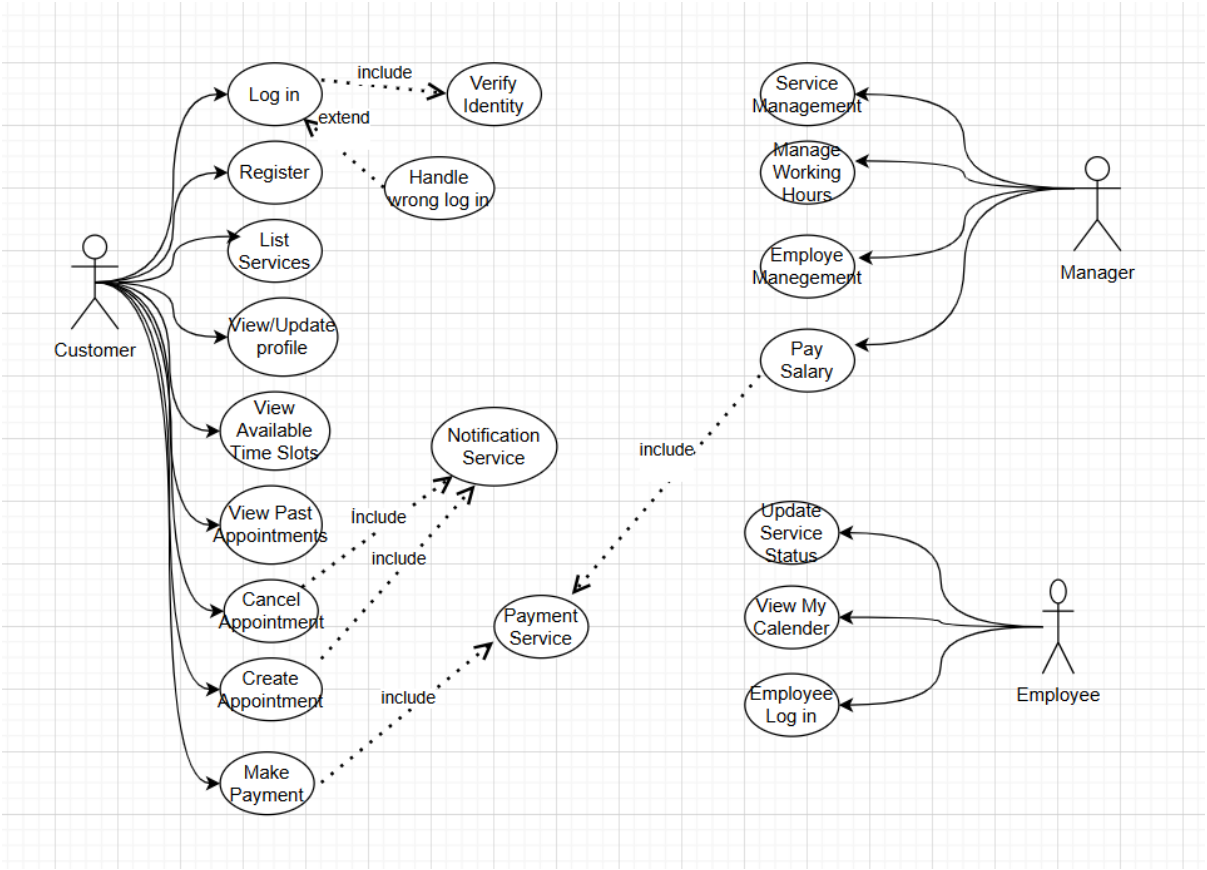



**Fig. 1.** Use case diagram illustrating core actor interactions and internal service dependencies.

The diagram highlights the specific functional boundaries and key relationships for each system actor. The **Customer** is responsible for self-service actions, including registration, logging in (which includes identity verification and extends wrong login handling), creating and canceling appointments (both of which include the internal Notification Service), and making payments (which includes the Payment Service). The **Employee** interacts with the system primarily to log in, view their personal calendar, and update the status of assigned services. Finally, the **Manager** oversees administrative workflows, including general service management, **employee management**, managing working hours, and initiating salary payouts (which structurally includes the Payment Service).

**TABLE I**

## 2.7 Requirements Traceability Matrix

| Requirement | Use Case | Service/Method | Test Case |
|---|---|---|---|
| Customer Registration | UC-01 (Register) | AuthService.registerCustomer | **TC-01** |
| System Login | UC-05 (Login) | AuthService.login | **TC-02** |
| Account Lockout | UC-05 (Lockout) | AuthService.login / User.registerFailedLogin | **TC-02** |

| | | | |
|---|---|---|---|
| Create Appointment | UC-01 (Create Appointment) | AppointmentService.createAppointment | **TC-03** |
| Appointment Notification | UC-01 (Create Appointment) | NotificationService.onAppointmentCreated | **TC-03** |
| Cancel Appointment | UC-03 (Cancel Appointment) | AppointmentService.cancelAppointment | **TC-04** |
| Service Payment | UC-02 (Make Payment) | PaymentService.processAppointmentPayment | **TC-05** |
| Salary Payment | UC-04 (Pay Salary) | PaymentService.paySalary | **TC-06** |
| Duplicate Salary Prevention | UC-04 (Pay Salary) | Employee.isSalaryPaidFor | **TC-06** |
| Update Service Status | UC-02 (Complete Service) | AppointmentService.updateStatusByEmployee | **TC-05** |

# 3. Phase 2 — Design

## 3.1 Introduction to Design Phase

This section constitutes the Phase 2 Report for the CenGIRL — Beauty Salon Management System, developed as part of the CME2210 Object Oriented Analysis and Design course at Dokuz Eylül University. Building upon the requirements analysis performed in Phase 1, this phase transforms those requirements into a structured and detailed system design.

The report covers the overall system architecture, a comprehensive class diagram, sequence diagrams for key use cases, statechart diagrams illustrating critical state transitions, and activity diagrams depicting the major process flows within the system.

## 3.2 Architectural Design (N-Tier Structure)

CenGIRL employs a strict Layered (N-Tier) architecture that enforces a clear separation of concerns. Every module in the system belongs to one of four well-defined layers, ensuring that business logic remains independent of user-interface concerns and data-access mechanisms.

### 3.2.1 Presentation Layer

The Presentation Layer is responsible for all console-based user interactions. It reads input from actors (Customer, Employee, Manager) and delegates every request to the Application Layer. This layer contains no business logic; it solely handles formatting, input parsing, and displaying results or error messages.

Key classes: ConsoleApplication, MainMenu, CustomerMenu, EmployeeMenu, ManagerMenu, ConsoleUI, ConsoleIO.

### 3.2.2 Application / Service Layer

The Application Layer coordinates use-case workflows by orchestrating domain objects and infrastructure services. Classes such as AppointmentService, PaymentService, and NotificationService reside here. This layer enforces authorization rules, validates inputs before passing them to the Domain Layer, and ensures transactional consistency.
The primary services and their responsibilities include:

- **AuthService:** Register, login, and lockout management.
- **AppointmentService:** Booking, cancellation, status updates, and slot queries.
- **PaymentService:** Appointment payments, salaries, and strategy dispatch.
- **ManagerService:** Service catalog and employee management.
- **UserProfileService:** Profile field updates.
- **NotificationService:** Observer callbacks and notification logging.

ApplicationContext acts as a composition root, wiring concrete infrastructure implementations to services.

### 3.2.3 Domain Layer

The Domain Layer is the core of the system and contains the rich business model. It houses all domain entities (User, Customer, Employee, Manager, Appointment, Service, Payment), value objects, and

domain-specific rules. All state transitions and business invariants are enforced here using strict encapsulation and access modifiers.

- Entities: User (abstract), Customer, Employee, Manager, Appointment, Payment, BeautyService
- Value objects: TimeSlot, WorkSchedule, EmployeeCalendar
- Enums: AppointmentStatus, PaymentStatus, PaymentType, UserRole
- Exceptions: DomainException hierarchy

### 3.2.4 Infrastructure Layer

The Infrastructure Layer provides in-memory repository implementations for all domain entities. Repositories use Java collection types (HashMap, HashSet) to ensure O(1) average-case lookup performance. The layer also includes helper utilities such as IdGenerator, DateTimeUtil, and InputValidator.

### 3.2.5 Component Interaction

The following dependency rules apply between layers:

- Presentation depends on Application only
- Application depends on Domain (interfaces) and Infrastructure utilities
- Infrastructure depends on Domain (implements repository interfaces)
- Domain depends on nothing — no imports from other layers

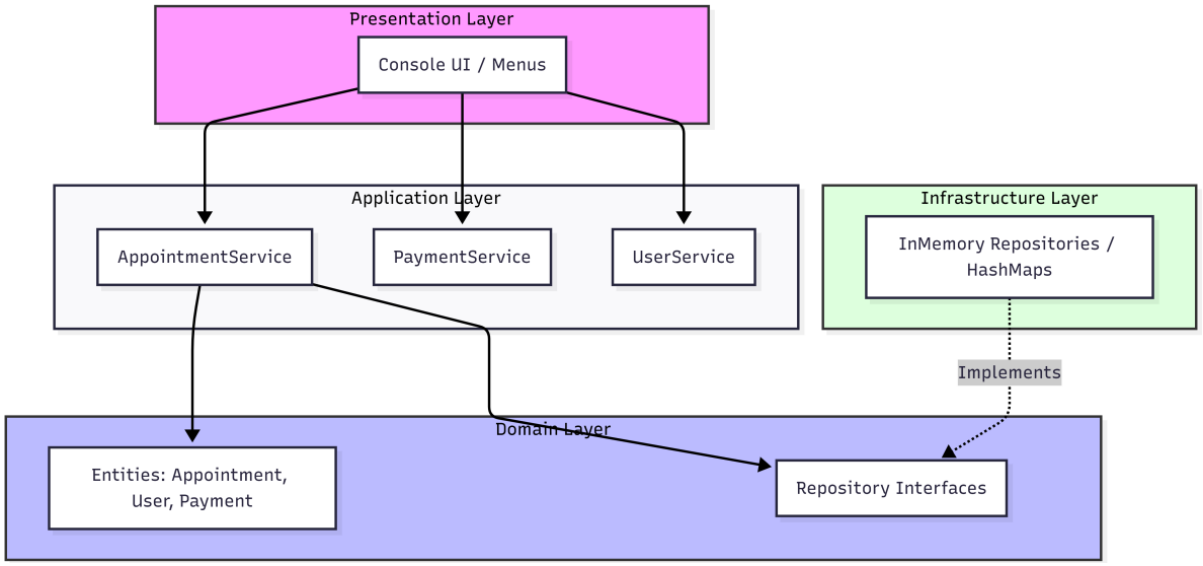



**Fig. 2**. Component interaction diagram illustrating strict layer dependencies and the implementation of repository interfaces.

## 3.3 Package Organization

The package structure mirrors the four-layer architecture, providing immediate navigability for reviewers:

```
com/
```

```
├── presentation/
│   ├── ConsoleApplication.java
│   ├── menu/     (MainMenu, CustomerMenu, EmployeeMenu, ManagerMenu)
│   └── util/     (ConsoleUI, ConsoleIO)
├── application/
│   ├── ApplicationContext.java
│   ├── SampleDataBootstrap.java
│   ├── service/ (Auth, Appointment, Payment, Manager,
│   │             Notification, UserProfile)
│   └── payment/ (PaymentStrategy + implementations)
├── domain/
│   ├── model/
│   ├── valueobject/
│   ├── repository/   (interfaces only)
│   ├── factory/
│   ├── observer/
│   ├── enums/
│   └── exception/
└── infrastructure/
    ├── repository/   (InMemory implementations)
    └── util/
```

**TABLE II**

**PACKAGE SUMMARY**

| Package | Number of Files | Purpose |
|---|---|---|
| **presentation** | 7 | Menus, console I/O helpers |
| **application** | 12 | Services, composition root, bootstrap |
| **domain** | 28 | Entities, value objects, interfaces, exceptions |
| **infrastructure** | 7 | In-memory repositories, utilities |
| **Total** | 54 | Java compilation units |

## 3.4 Design Patterns Applied

Design patterns are recurring solutions to common object-oriented design problems. In CenGIRL they are not decorative labels; each pattern addresses a **specific extensibility or**

**coupling problem** identified during Phase 1 (payments, user roles, notifications, persistence) and is realized concretely in Phase 3 Java code.

**Table III**
**Summary of Applied Design Patterns**

| Pattern | Category | Problem Solved | Intent | Location in Code |
|---|---|---|---|---|
| **Repository** | Structural / Persistence | Services must not depend on HashMap or SQL details | Abstract persistence behind an interface | `com.domain.repository.*` + `com.infrastructure.repository.InMemory*` |
| **Strategy** | Behavioral | Multiple payment methods without `if-else` chains in `PaymentService` | Interchangeable payment algorithms at runtime | `PaymentStrategy`, `CashPaymentStrategy`, `CreditCardPaymentStrategy`, `DigitalWalletPaymentStrategy` |
| **Factory Method** | Creational | `AuthService` should not contain `new Customer()` / `new Employee()` logic | Create role-specific users without coupling services to concrete types | `UserFactory.create()` — dispatches on `UserRole` enum |
| **Observer** | Behavioral | Appointment service should not print notifications or send email itself | React to appointment lifecycle events | `AppointmentObserver`, `NotificationService` |
| **Composition Root** | Architectural | Avoid scattering `new InMemory*Repository()` across menus | Centralized object graph wiring | `ApplicationContext` |

The subsections below explain **why** each pattern was chosen, **how** classes collaborate, and **which use cases** trigger them.

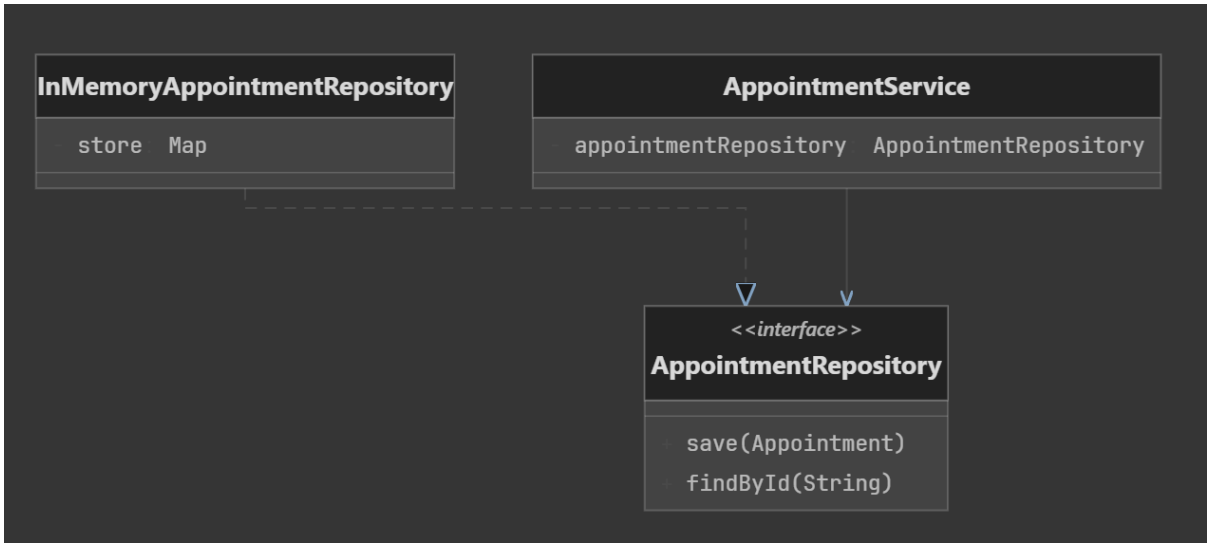
### 3.4.1 Repository Pattern

**Problem**
`AppointmentService` and `PaymentService` need to load and store domain objects. If they called `HashMap.put` directly, three problems would appear:

1. **Layer violation** — application logic would depend on infrastructure details.
2. **Poor testability** — swapping in-memory storage for a database would require rewriting every service.
3. **Scattered persistence rules** — duplicate save/find logic across services.

**Solution**

Define **repository interfaces in the Domain layer** and implement them in the **Infrastructure layer**. Services depend only on interfaces (`AppointmentRepository`, `UserRepository`, etc.).




**Participants**

| Role | Class |
|---|---|
| **Repository interface** | `AppointmentRepository`, `UserRepository`, `BeautyServiceRepository`, `PaymentRepository` |
| **Concrete repository** | `InMemoryAppointmentRepository`, `InMemoryUserRepository`, … |
| **Client** | `AppointmentService`, `PaymentService`, `AuthService`, `ManagerService` |
| **Aggregate** | `Appointment`, `User`, `BeautyService`, `Payment` |

**Collaboration (example: cancel appointment)**

1. `AppointmentService.cancelAppointment` loads `Appointment` via `appointmentRepository.findById`.
2. Domain method `appointment.cancel()` mutates state.
3. Service calls `appointmentRepository.save(appointment)` — same object reference is updated in the `HashMap`.
4. No service knows that storage is a `HashMap`; only `InMemoryAppointmentRepository` does.

**Code contract (domain interface)**

```java
public interface AppointmentRepository {
    void save(Appointment appointment);
    Optional<Appointment> findById(String id);
    List<Appointment> findByCustomerId(String customerId);
    List<Appointment> findByEmployeeId(String employeeId);
    List<Appointment> findAll();
}
```

**Infrastructure implementation**
`InMemoryAppointmentRepository` uses `Map<String, Appointment>` for **O(1)** lookup by primary key. Filtered queries (`findByCustomerId`) scan values — acceptable for in-memory academic scale.

**Benefits and SOLID link**

- **DIP (Dependency Inversion):** High-level `AppointmentService` depends on abstraction, not `HashMap`.
- **OCP (Open-Closed):** Add `JdbcAppointmentRepository` without changing service signatures.
- **SRP:** Persistence mechanics isolated in one class per aggregate.

**Future extension**
A database-backed repository would implement the same interface; `ApplicationContext` would wire the new implementation once — zero changes to domain entities.

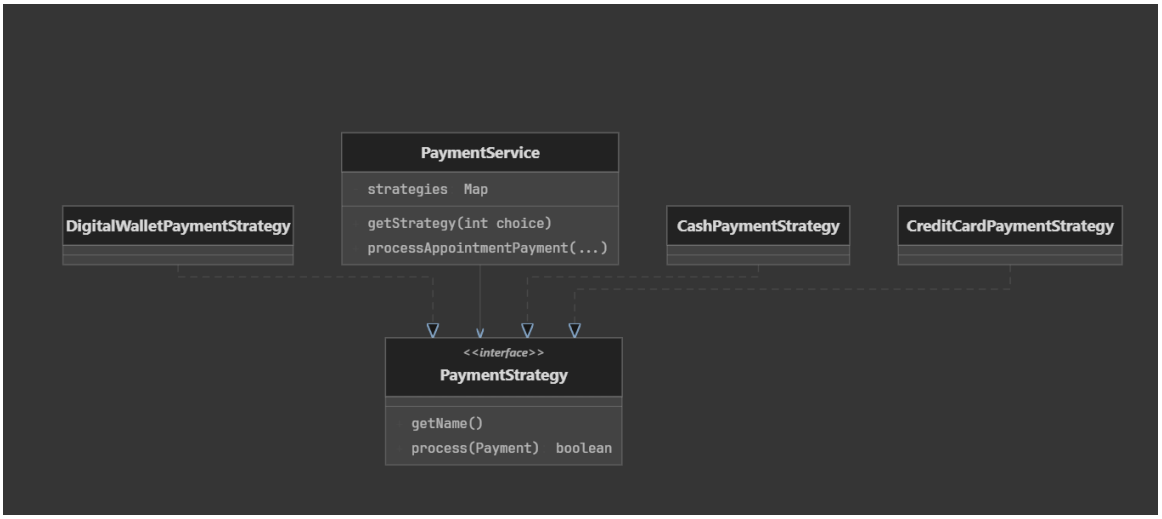
### 3.4.2 Strategy Pattern — Payment

**Problem**
Customers and managers pay through **cash**, **credit card**, or **digital wallet**. Business rules differ per method (e.g., card may reject amounts above a limit). Putting all logic inside `PaymentService.processAppointmentPayment` as a large `switch` would:

- Violate **Open-Closed Principle** (every new method edits the service).
- Mix orchestration with low-level payment rules.
- Complicate unit testing of individual methods.

**Solution**

Define a common interface `PaymentStrategy` with `process(Payment)` and `getName()`. `PaymentService` selects a strategy at runtime based on user menu input.




**Participants**

| Role | Class |
|---|---|
| **Context** | `PaymentService` — holds strategy map, invokes selected strategy |
| **Strategy interface** | `PaymentStrategy` |
| **Concrete strategies** | `CashPaymentStrategy`, `CreditCardPaymentStrategy`, `DigitalWalletPaymentStrategy` |
| **Data passed in** | `Payment` domain entity (amount, status) |

**Registration and selection (Composition Root + Strategy)**
At construction time, `PaymentService` registers strategies:

```
strategies.put(1, new CashPaymentStrategy());
strategies.put(2, new CreditCardPaymentStrategy());
strategies.put(3, new DigitalWalletPaymentStrategy());
```

When the customer chooses option `2` in the console menu, `getStrategy(2)` returns `CreditCardPaymentStrategy`.

**Behavior per concrete strategy**

| Strategy | `getName()` | `process(Payment)` behavior (simulated) |
|---|---|---|
| **Cash** | `"Cash"` | Always returns `true` (cash accepted locally) |
| **Credit Card** | `"Credit Card"` | Returns `true` only if amount ≤ 50,000 TRY (simulated limit) |
| **Digital Wallet** | `"Digital Wallet"` | Returns `true` only if amount > 0 |

These differences demonstrate **runtime algorithm variation** without changing `PaymentService`'s overall flow.

**Collaboration sequence (appointment payment)**

1. `PaymentService.processAppointmentPayment` validates customer ownership and `COMPLETED` status.
2. Pending `Payment` is created or retrieved.
3. `PaymentStrategy strategy = getStrategy(methodChoice);`
4. `boolean success = strategy.process(payment);`
5. If success: `payment.complete(strategy.getName())` and `appointment.markPaid()`; else `payment.fail()`.
6. Repositories persist updated entities

.

The **same strategy objects** are reused for **manager salary payments** (`paySalary`), proving the pattern generalizes beyond customer checkout.

**Benefits and SOLID link**

- **OCP:** New `BankTransferPaymentStrategy` = new class + one line in strategy map; no change to `Payment.complete()`.
- **SRP:** `PaymentService` orchestrates; each strategy owns method-specific rules.
- **LSP:** Any `PaymentStrategy` implementation can substitute another at runtime.

**Use cases triggered**

- **UC-02** Complete Service and Make Payment (customer path).
- **UC-04** Pay Salary (manager path).

### 3.4.3 Factory Method Pattern — User Creation

**Problem**

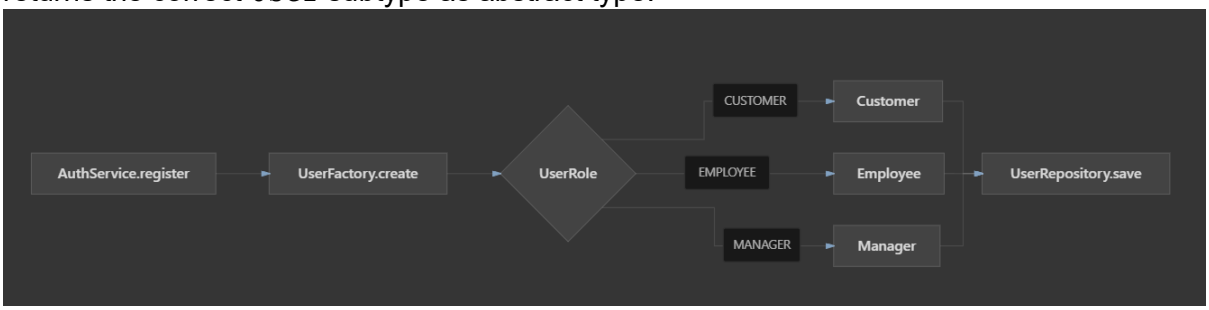
The system has three user subtypes (`Customer`, `Employee`, `Manager`) with **different constructor parameters**:

- `Customer` — basic profile only.
- `Employee` — requires `WorkSchedule` and `BigDecimal monthlySalary`.
- `Manager` — administrative user without calendar.

If `AuthService.register` contained `new Customer(...)`, `new Employee(...)`, etc., registration logic would be **duplicated** and **coupled** to concrete classes. Any new field on `Employee` would force edits in every caller.

**Solution**

Centralize creation in `UserFactory.create(UserRole role, ...)` using a **switch expression** on `UserRole`. Callers pass role and parameters; the factory returns the correct `User` subtype as abstract type.



**Participants**

| Role | Class |
|---|---|
| **Creator** | `AuthService` (calls factory; does not instantiate subtypes directly) |
| **Factory** | `UserFactory` (static `create` method) |
| **Products** | `Customer`, `Employee`, `Manager` |
| **Abstract product** | `User` |

**Code (factory dispatch)**

```
public static User create(UserRole role, String id, String username,
...) {
    return switch (role) {
        case CUSTOMER -> new Customer(id, username, password, fullName,
email, phone);
        case EMPLOYEE -> new Employee(id, username, password, fullName,
email, phone, schedule, salary);
        case MANAGER -> new Manager(id, username, password, fullName,
email, phone);
    };
}
```

`AuthService.registerCustomer` calls `register(UserRole.CUSTOMER, ...)`, which delegates to `UserFactory`. `ManagerService.addEmployee` calls `authService.register(UserRole.EMPLOYEE, ...)` — **same factory**, different entry point.

**Why this is Factory Method (not Simple Factory only)**

- Creation logic is **encapsulated** in a dedicated factory class with private constructor (`UserFactory` cannot be instantiated).
- The **product family** shares `User` interface; services work with `User` until role-specific behavior is required (`instanceof Employee` in calendar operations).
- Extending with a new role (e.g., `RECEPTIONIST`) means extending the switch and adding one product class — callers of `UserFactory.create` remain stable.

**Benefits and SOLID link**

- **SRP:** `AuthService` handles validation and persistence; `UserFactory` handles object construction.
- **OCP:** New user type = new class + factory branch (minimal, localized change).
- **DIP:** Services depend on `User`, not concrete constructors scattered in presentation menus.

**Use cases triggered**

- Customer registration (**UC-05** / FR-C-01).
- Manager adds employee (**FR-M-02**).
- Bootstrap seed data in `SampleDataBootstrap`.

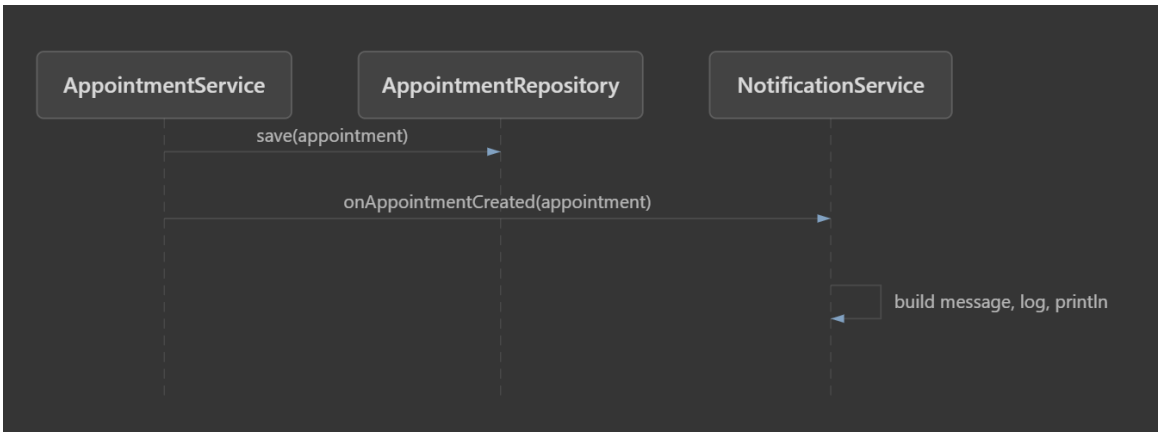
### 3.4.4 Observer Pattern — Notifications

**Problem**
When an appointment is **created** or **cancelled**, both customer and employee should be informed. Embedding `System.out.println` or future email/SMS code **inside** `AppointmentService.createAppointment` would:

- Bloat the appointment workflow.
- Make it hard to add second observers (e.g., analytics, audit log).
- Violate **Single Responsibility** — appointment orchestration vs. notification delivery.

**Solution**

Define subject events as methods on `AppointmentObserver`. `NotificationService` implements the interface. `AppointmentService` invokes the observer **after** successful persistence.




**Participants**

| Role | Class |
|---|---|
| **Subject (implicit)** | `AppointmentService` — emits events after state changes |
| **Observer interface** | `AppointmentObserver` |
| **Concrete observer** | `NotificationService` |
| **Event payload** | `Appointment` entity (id, customerId, employeeId, timeSlot) |

**Observer contract**

```
public interface AppointmentObserver {
    void onAppointmentCreated(Appointment appointment);
    void onAppointmentCancelled(Appointment appointment);
}
```

**Concrete observer behavior**
`NotificationService`:

1. Builds a human-readable message from appointment data.
2. Appends to `notificationLog` (in-memory list for demo/audit).
3. Prints to console (stand-in for SMS/email in Phase 3 scope).

Example message on create:
`[NOTIFICATION] Booking confirmed for appointment APT-... | customer=..., employee=..., slot=...`

**When notifications fire**

| Event | Trigger method | Preconditions |
|---|---|---|

| | | |
|---|---|---|
| Created | `onAppointmentCreated` | After `save`, slot booked, customer linked |
| Cancelled | `onAppointmentCancelled` | After `cancel()`, slot released, `save` |

Notifications are **not** sent if validation fails earlier (slot unavailable, wrong customer) — observer runs only on successful commits.

**Extensibility**
Additional observers (e.g., `AuditLogObserver`, `EmailObserver`) could be registered in a list inside `AppointmentService` without modifying `NotificationService`. Current design uses a single injected `NotificationService` for simplicity; a `List<AppointmentObserver>` would be the natural next step.

**Benefits and SOLID link**

- **SRP:** `AppointmentService` coordinates; `NotificationService` formats and delivers messages.
- **OCP:** New channel = new observer class.
- **DIP:** `AppointmentService` depends on `NotificationService` / observer abstraction, not console APIs.

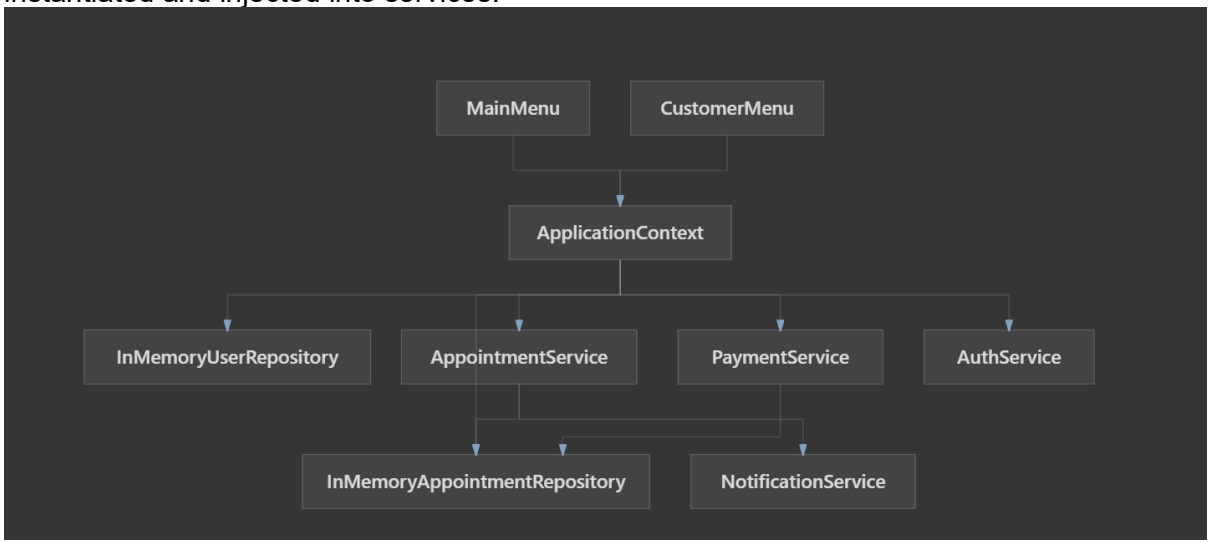
**Use cases triggered**

- **UC-01** Create Appointment (included: Notification Service).
- **UC-03** Cancel Appointment (included: Notification Service).

### 3.4.5 Composition Root — ApplicationContext

**Problem**
Constructing repositories and services in every menu class would duplicate wiring and hide the **dependency graph**. Presentation layer must never choose between `InMemoryUserRepository` vs. a future JDBC implementation.

**Solution**
`ApplicationContext` acts as the **composition root** (Mark Seemann / Clean Architecture term): the single place in the application layer where concrete infrastructure types are instantiated and injected into services.




**Wiring excerpt**

```
private final UserRepository userRepository = new
InMemoryUserRepository();
private final AppointmentRepository appointmentRepository = new
InMemoryAppointmentRepository();
private final NotificationService notificationService = new
NotificationService();
private final AppointmentService appointmentService = new
AppointmentService(
        appointmentRepository, beautyServiceRepository, userRepository,
notificationService);
```

Constructor also runs `SampleDataBootstrap.seed(this)` so demo data exists before menus display.

**Rules enforced**

1. **Only** `ApplicationContext` references `com.infrastructure.repository.InMemory*`.
2. Menus call `context.getAppointmentService()`, never `new AppointmentService(...)`.
3. Domain layer never imports `ApplicationContext`.

**Relation to other patterns**

- **Repository:** Context chooses `InMemory*` implementations.
- **Strategy:** Context builds `PaymentService`, which internally registers payment strategies.
- **Observer:** Context creates one `NotificationService` shared by `AppointmentService`.

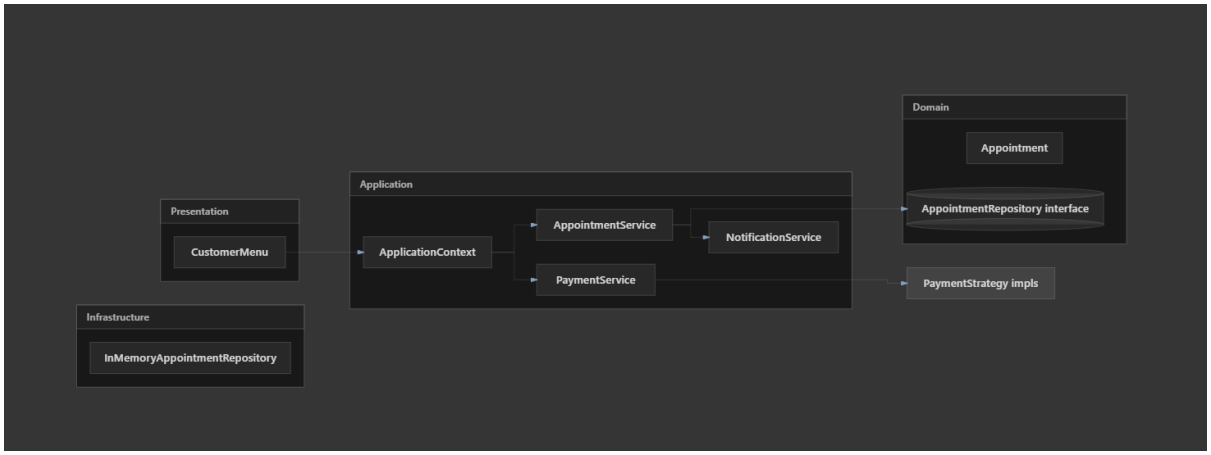
**Benefits**

- Clear **object graph** for reviewers and documentation.
- One-line swap for integration testing (mock context or test-specific context subclass).
- Prevents **service locator anti-pattern** in UI — context is passed once to `MainMenu`, not looked up globally.

### 3.4.6 Pattern Interaction Overview

The patterns work together in typical flows:

| Flow | Patterns involved |
|---|---|
| Customer books appointment | Repository (save), Observer (notify), Composition Root (wiring) |
| Customer pays | Repository (load/save), Strategy (payment method), domain state on `Payment`/`Appointment` |
| Manager registers employee | Factory Method (create User), Repository (save), Composition Root |
| Manager pays salary | Strategy + Repository + Factory-created `Employee` |




### 3.4.7 Patterns Considered but Not Used (Brief)

| Pattern | Why not chosen |
|---|---|
| **Singleton** | `ApplicationContext` is one instance per `main`, but not enforced globally — avoids hidden global state. |
| **MVC** | Console menus are thin; full MVC adds little over current menu + service split. |
| **Decorator** | Payment strategies already cover varying behavior. |
| **Command** | Menu `switch` is sufficient for console scale; could refactor if undo/redo needed. |

## 3.5 Class Diagram

### 3.5.1 Key Classes and Relationships

**TABLE IV**
KEY CLASSES AND THEIR RELATIONSHIPS

| Class | Responsibility | Key Relationships |
|---|---|---|
| **User (abstract)** | Base class for all actors; holds credentials and profile data | Inherited by Customer, Employee, Manager |

| | | |
|---|---|---|
| **Customer** | Manages appointments, views services, makes payments | Aggregates List<Appointment IDs> |
| **Employee** | Manages calendar, updates service statuses | Aggregates EmployeeCalendar; associated with List<Appointment> |
| **Manager** | Controls service catalog, employee records, payroll | Aggregates List<Employee>, List<Service> |
| **Appointment** | Central domain entity; holds service, time slot, and status | Associated with Customer, Employee, Service, Payment |
| **BeautyService** | Represents a beauty service (name, duration, price) | Referenced by Appointment |
| **Payment** | Tracks financial transactions for appointments and salaries | Associated with Appointment or Employee |
| **AppointmentService** | Use-case orchestration for appointment CRUD | Depends on AppointmentRepository, NotificationService |
| **PaymentService** | Processes customer payments and salary payments | Uses Strategy pattern for payment methods |
| **NotificationService** | Sends notifications on appointment events | Observer of Appointment lifecycle |

### 3.5.2 Class Diagram Visualization

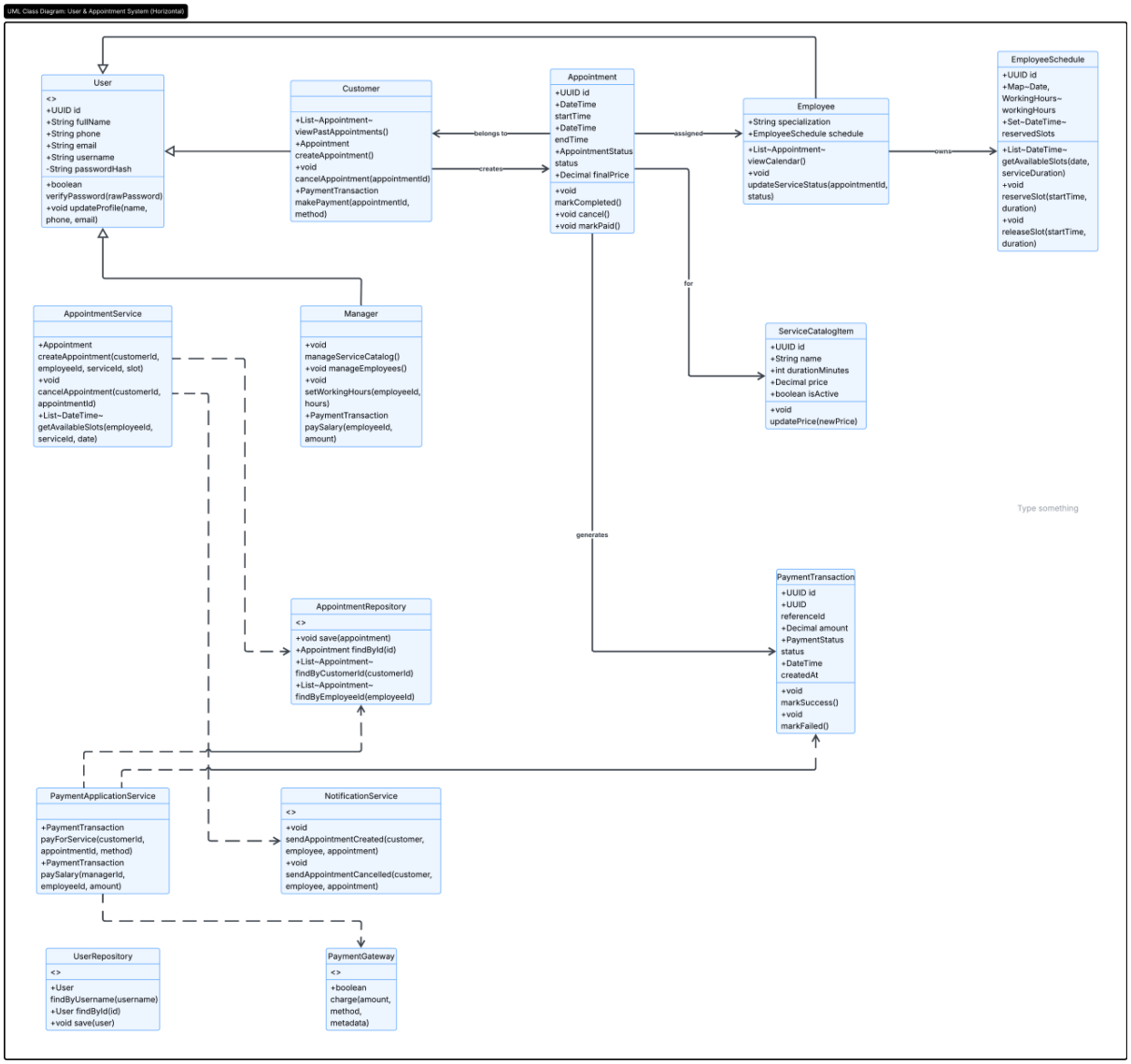



**Fig. 3.** Core domain model and service layer class diagram.

The class diagram captures all major domain entities, their attributes, methods, and the relationships between them (inheritance, association, aggregation, and dependency). Key structural decisions reflected in the diagram include:

- User is abstract; all three actor types inherit shared credential and profile behavior
- Appointment references Customer, Employee, and BeautyService by ID (loose coupling through identity)
- EmployeeCalendar is a value object owned by Employee, not a separate entity
- PaymentStrategy is an interface; concrete implementations are in the application layer
- Repository interfaces are declared in domain; implementations reside in infrastructure

## 3.6 Sequence Diagrams

Sequence diagrams illustrate the interactions between objects over time for specific use cases. The four diagrams below correspond to the most critical workflows in the system.

### 3.6.1 Create Appointment

This sequence captures the end-to-end flow when a customer creates a new appointment. The customer selects a service and employee via the Presentation Layer, which delegates to AppointmentService. The

service validates availability through AppointmentRepository, persists the new Appointment entity, and triggers NotificationService to dispatch a booking confirmation to both the customer and the assigned employee.

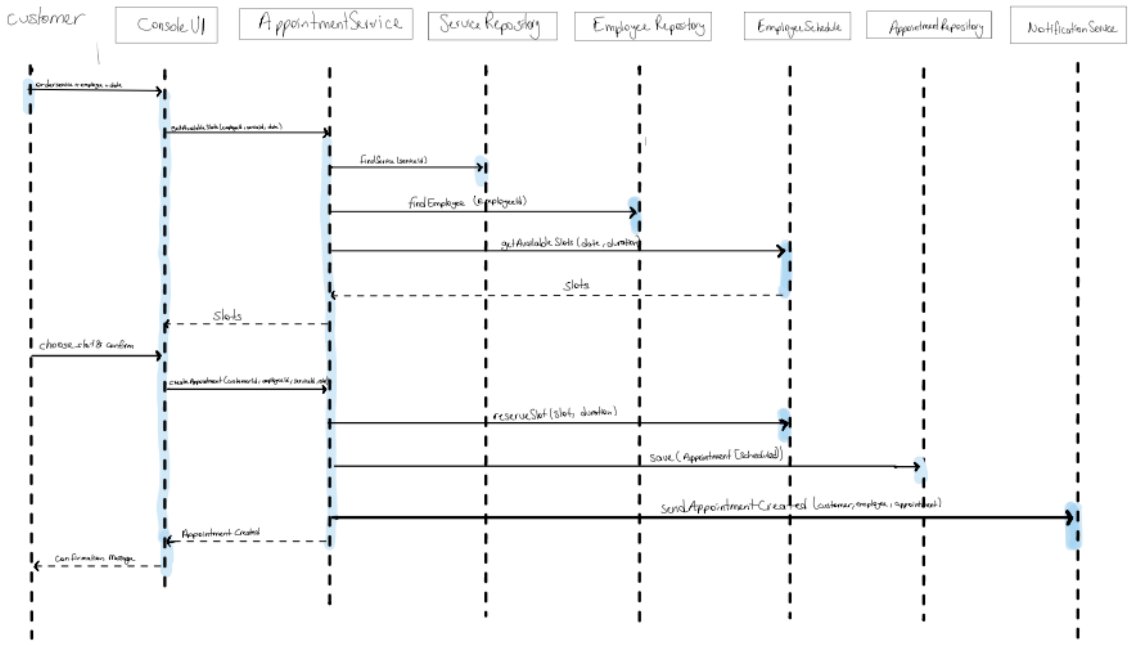



**Fig. 4.** Create Appointment Sequence Diagram.

Message flow summary:

- Customer → ConsoleUI: selectService, selectEmployee, selectSlot
- ConsoleUI → AppointmentService: createAppointment(customerId, employeeId, serviceId, slot)
- AppointmentService → ServiceRepository: findService(serviceId)
- AppointmentService → EmployeeRepository: findEmployee(employeeId)
- AppointmentService → EmployeeSchedule: getAvailableSlots(date, duration)
- AppointmentService → AppointmentRepository: save(appointment)
- AppointmentService → NotificationService: sendAppointmentCreated(customer, employee, appointment)

### 3.6.2 Cancel Appointment

When a customer requests a cancellation, AppointmentService first verifies that the appointment is in an active (Scheduled) state. It then updates the appointment status to Cancelled, releases the time slot in the employee's Calendar, and notifies both parties via NotificationService.

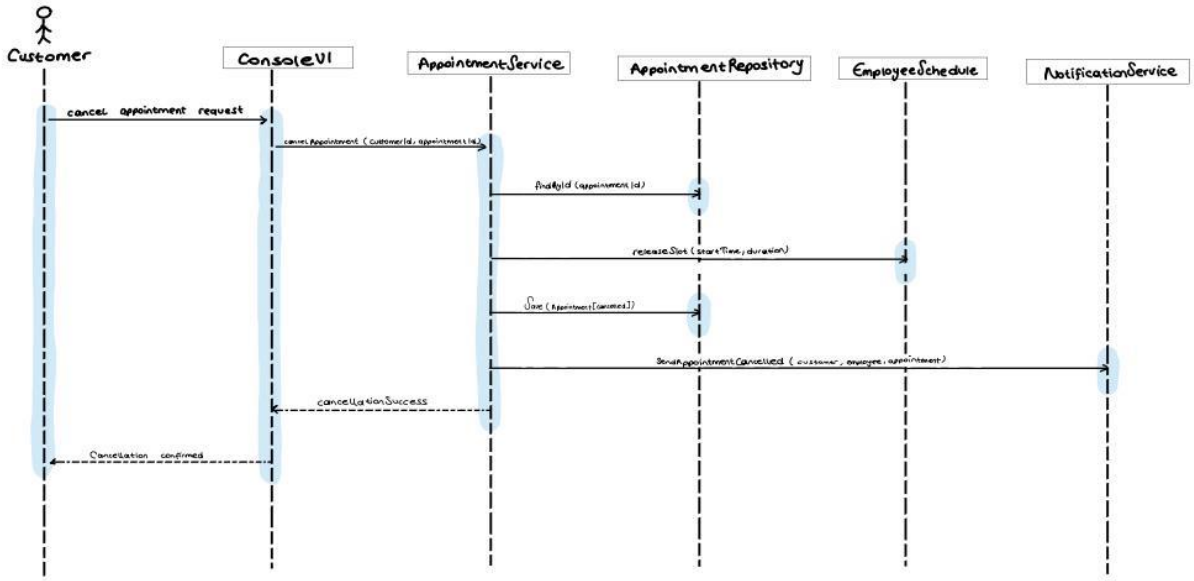



**Fig. 5.** Cancel Appointment Sequence Diagram.

Message flow summary:

- Customer → AppointmentService: cancelAppointment(customerId, appointmentId)
- AppointmentService → AppointmentRepository: findById(appointmentId)
- AppointmentService → EmployeeSchedule: releaseSlot(startTime, duration)
- AppointmentService → AppointmentRepository: save(appointment [cancelled])
- AppointmentService → NotificationService: sendAppointmentCancelled(customer, employee, appointment)

### 3.6.3 Complete Service and Make Payment

This two-part sequence begins with the Employee marking a scheduled appointment as Completed, which triggers the creation of a pending Payment record. Subsequently, the Customer accesses the payment interface, and the PaymentService processes the transaction using the selected payment strategy. On success, the appointment status is updated to Paid.

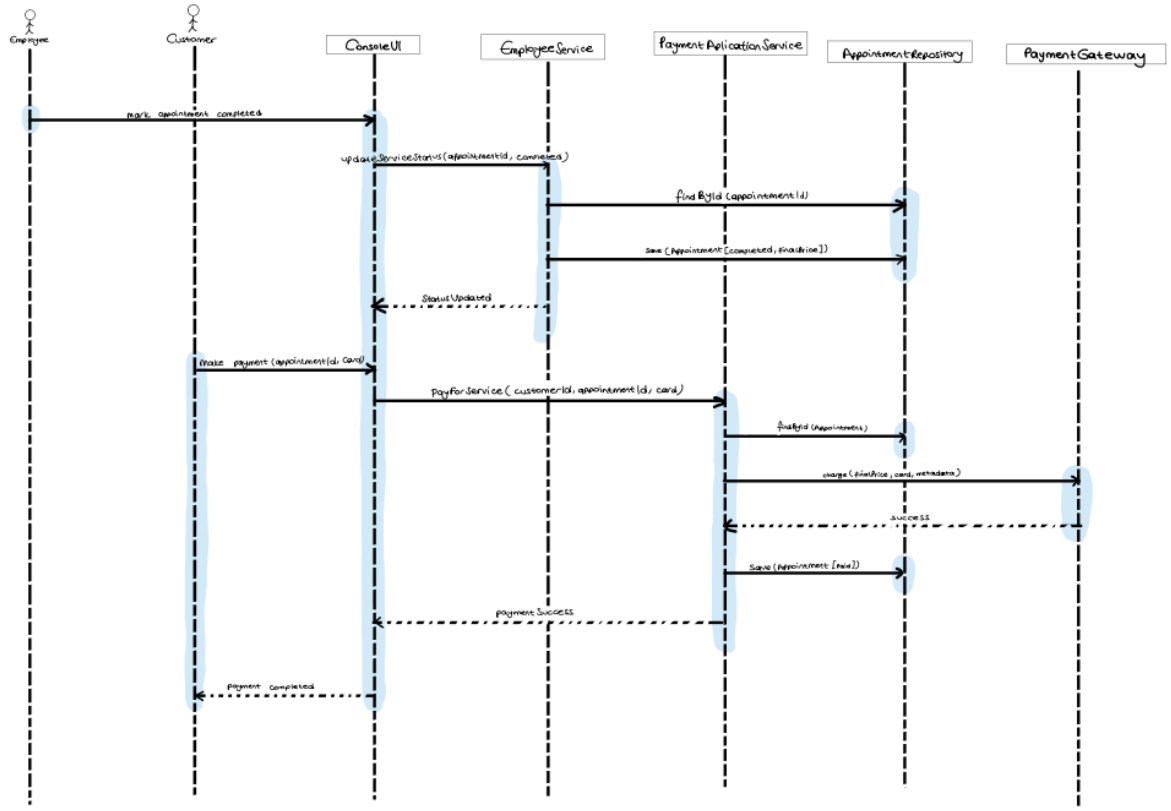



**Fig. 6.** Complete Service and Make Payment Sequence Diagram.

The diagram covers two actors in sequence: Employee marks completion first, then Customer initiates payment. The PaymentService uses the Strategy pattern to delegate processing to the chosen payment method implementation.

### 3.6.4 Pay Salary

The Manager selects an employee from the payroll module and confirms the monthly salary amount. The PaymentService processes the transaction and updates the Employee's financial record to Paid for the current period. A confirmation message is returned to the manager.

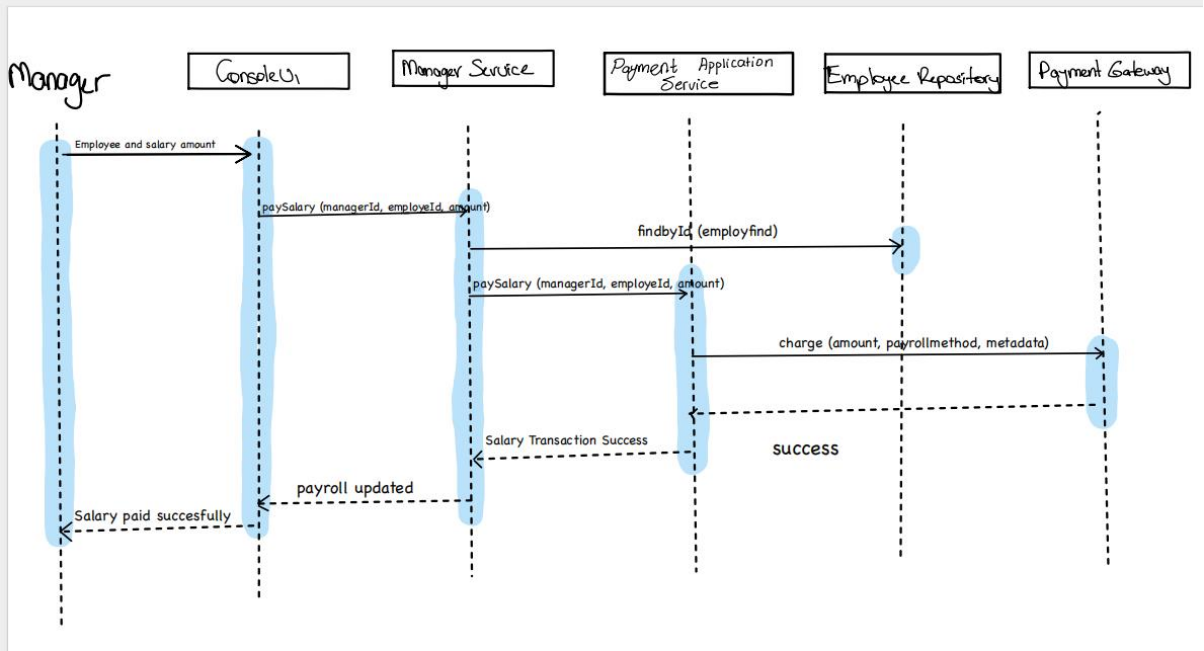



**Fig. 7.** Pay Salary Sequence Diagram.

## 3.7 Statechart Diagrams

Statechart diagrams model the lifecycle of critical domain objects by capturing all possible states and the transitions between them. Two objects with rich state behaviour are selected: the Appointment entity and the Payment entity.

### 3.7.1 Appointment Lifecycle

An Appointment begins in the Scheduled state immediately upon creation. It can transition to Completed when the employee marks the service as done, to Cancelled when the customer requests cancellation, or to NoShow if the customer does not attend. From Completed, the appointment moves to Paid once the customer's payment is successfully processed.

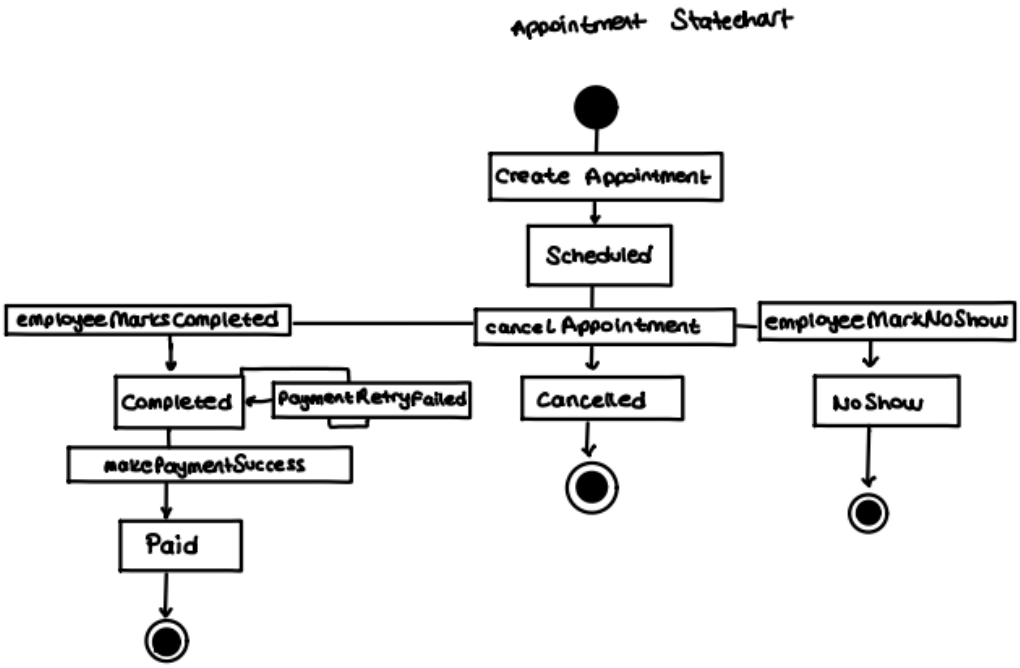



**Fig. 8.** Appointment Lifecycle Statechart Diagram.

**TABLE V**

**APPOINTMENT STATE TRANSITIONS**

| Current State | Trigger | Next State |
|---|---|---|
| (initial) | Customer creates appointment | Scheduled |
| Scheduled | Employee marks service done | Completed |
| Scheduled | Customer cancels | Cancelled |
| Scheduled | Customer does not attend | NoShow |
| Completed | Customer payment successful | Paid |
| Completed | Payment retry failed (business rule) | Remains Completed |

Implementation note: Appointment.cancel() allows direct transition to Cancelled from Scheduled without using the generic transition() helper — by design for cancellation semantics. The markPaid() method requires COMPLETED as precondition; calling it on SCHEDULED throws InvalidStateTransitionException.

### 3.7.2 Payment Lifecycle

A Payment record is created in a Pending state. If the transaction is processed successfully by the PaymentService, the record transitions to Completed. If the transaction fails (e.g., due to insufficient funds or a system error), it transitions to Failed, which allows for a retry. A Completed payment may be refunded, resulting in a Refunded terminal state.

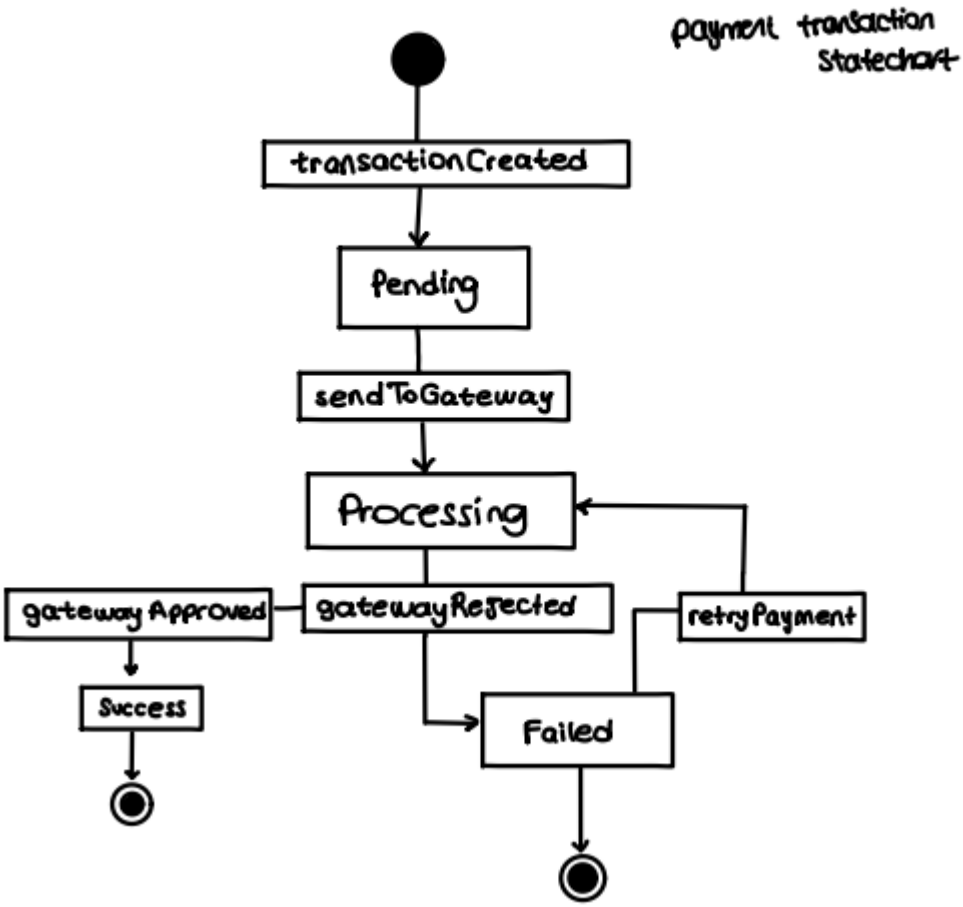



**Fig. 9.** Payment Lifecycle Statechart Diagram.

**TABLE VI**

**PAYMENT STATE TRANSITIONS**

| Current State | Trigger | Next State |
|---|---|---|
| (initial) | Payment created | Pending |
| Pending | sendToGateway | Processing |
| Processing | gatewayApproved | Success (Completed) |
| Processing | gatewayRejected | Failed |
| Failed | retryPayment | Processing |
| Completed | refund() | Refunded |

## 3.8 Activity Diagrams

Activity diagrams model the sequential and concurrent flow of activities within a process, including decision points and parallel paths. Three key workflows are represented.

### 3.8.1 User Login and Authentication

This activity diagram covers the authentication flow common to all actors. The user enters their credentials; the system verifies them against the in-memory repository. If verification fails, the system increments the failed-attempt counter. After a configurable number of consecutive failures (threshold = 3), the account is temporarily locked. On successful verification, the system loads the role-specific dashboard.

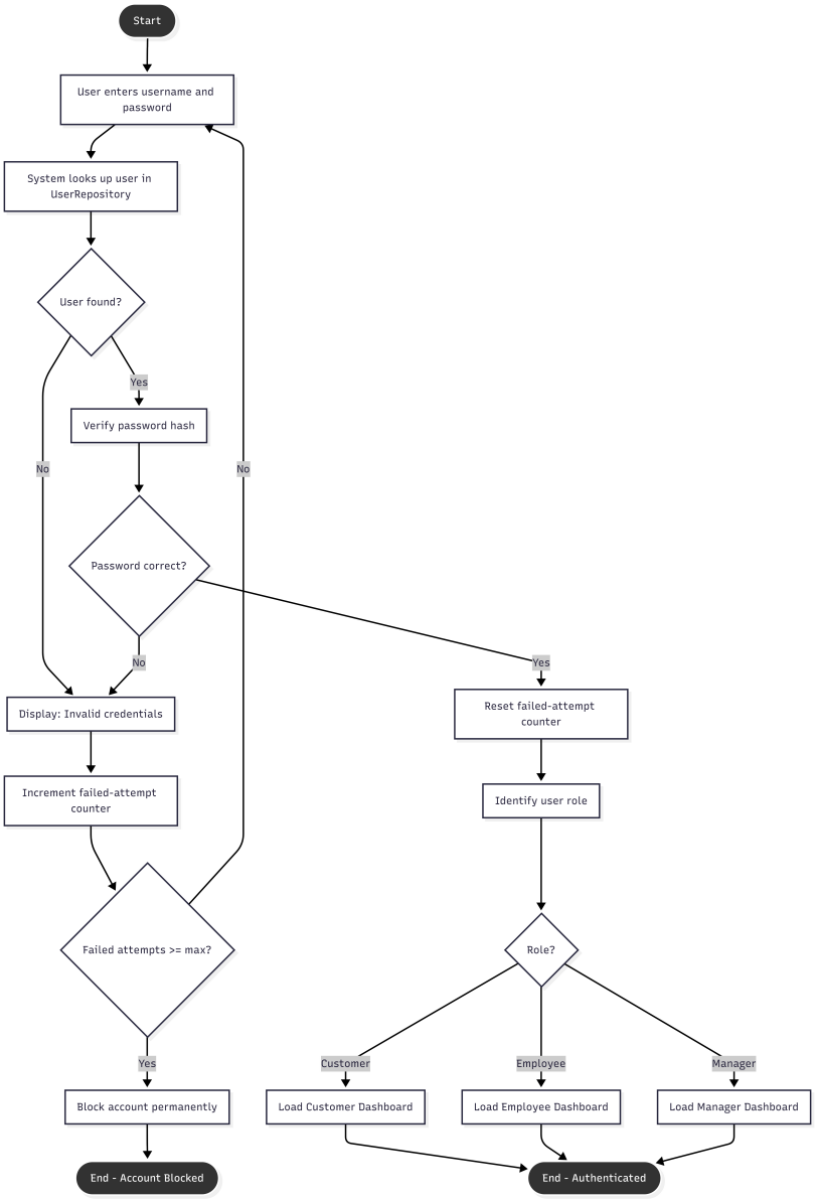



**Fig. 10.** User Login and Authentication Activity Diagram.

Decision points: User Found? → Password Correct? → Failed Attempts >= Max? → Role dispatch (Customer / Employee / Manager)

### 3.8.2 Appointment Booking Process

The booking workflow starts when an authenticated customer navigates to the appointment creation screen. The customer selects a service and an employee; the system fetches available time slots from

the employee's Calendar. After the customer confirms the booking, the system checks for concurrent conflicts (double-booking guard), persists the appointment, updates the employee's calendar, and dispatches confirmation notifications to both parties.

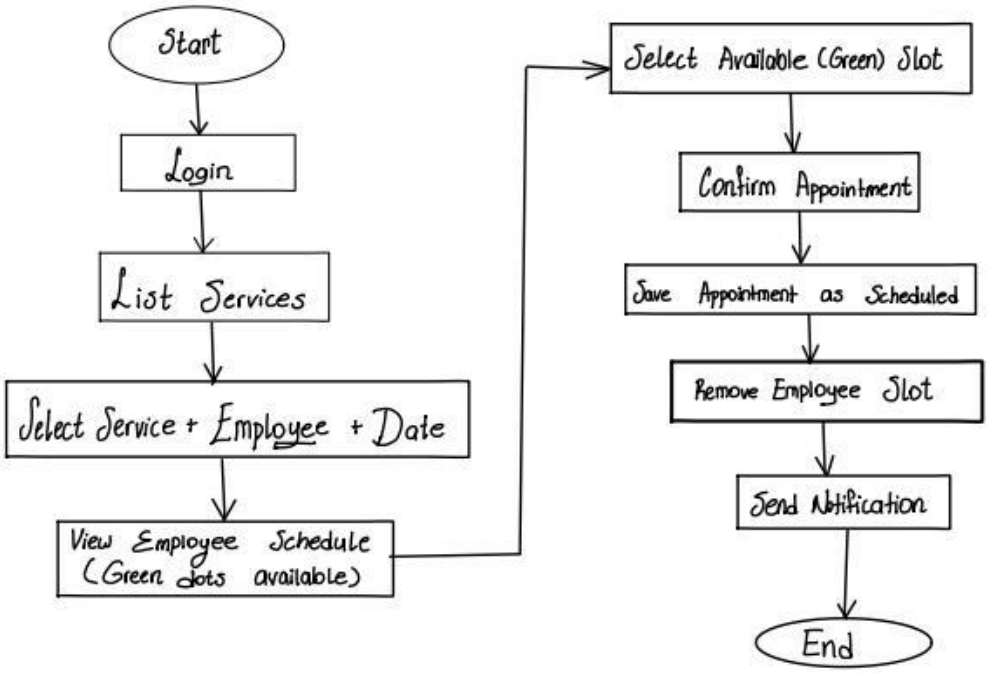



**Fig. 11.** Appointment Booking Process Activity Diagram.

### 3.8.3 Payment Processing

When a customer initiates payment for a completed appointment, the system presents the available payment methods (Strategy pattern). The customer selects a method and confirms the amount. The PaymentService executes the transaction; if it succeeds, the appointment is marked Paid and a receipt is generated. If it fails, the customer is prompted to retry or choose an alternative payment method.

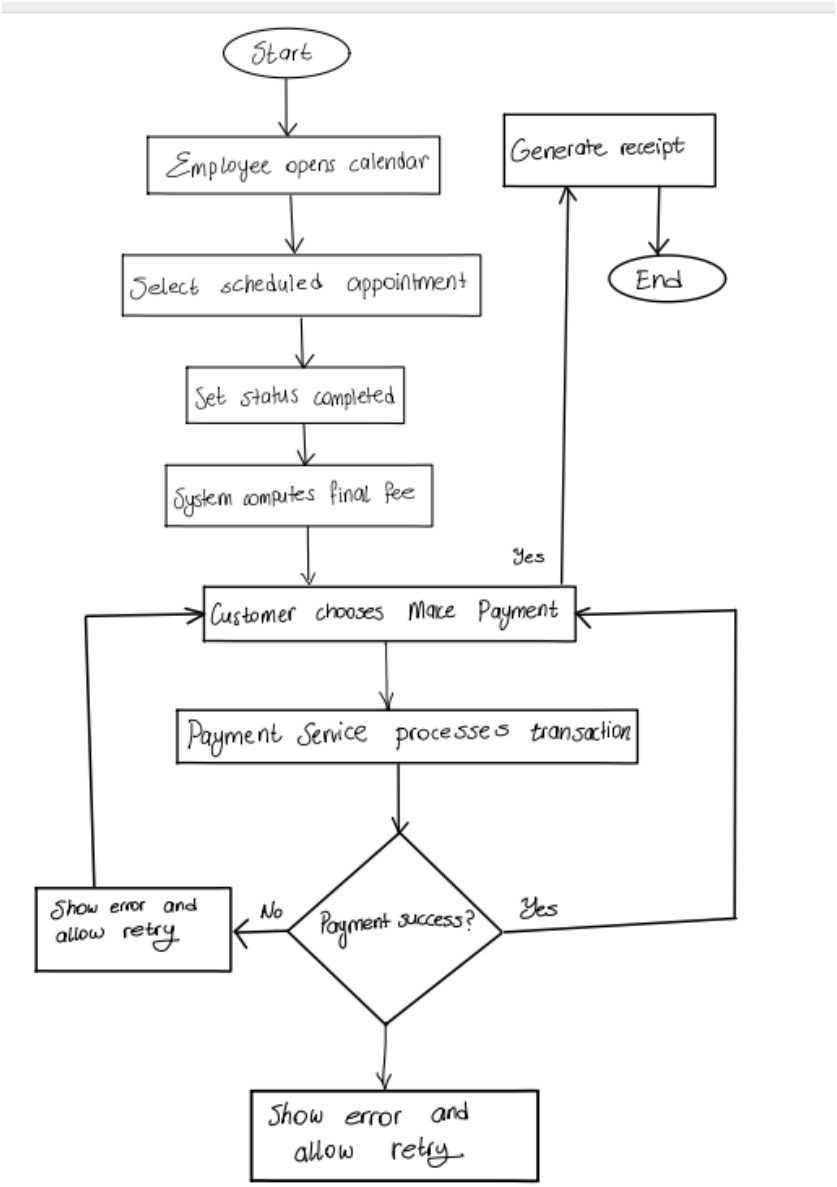



**Fig. 11.** Payment Process Activity Diagram.

## 3.9 Overall System Design Description

The CenGIRL Beauty Salon Management System is designed around a four-layer architecture that strictly enforces separation of concerns. The Presentation Layer interacts with three distinct actor types — Customer, Employee, and Manager — directing each request through role-based access controls implemented in the Application Layer.

The Application Layer acts as the orchestration hub. AppointmentService coordinates the complete appointment lifecycle: creation, status updates, and cancellation. PaymentService processes both customer-facing and internal (salary) financial transactions using the Strategy design pattern, which allows payment methods to be added or changed without modifying the core service logic. NotificationService operates as an observer, automatically dispatching messages whenever significant appointment events occur.

All business rules and domain invariants are enforced exclusively within the Domain Layer. The Appointment entity is the central domain object; its state machine (Scheduled → Completed/Cancelled/NoShow → Paid) is protected through strict encapsulation, preventing any layer above from illegally mutating state without following the defined transition rules. Similarly, the Calendar value object associated with each Employee is exclusively managed by AppointmentService, guaranteeing that no double-booking can occur.

The Infrastructure Layer backs all domain repositories with HashMap-based in-memory stores, ensuring consistent O(1) access for appointment lookups by ID and O(n) for filtered queries. All repositories are accessed through interfaces, making it straightforward to substitute persistent database implementations in a future phase.

Overall, the design achieves high cohesion within each layer and low coupling between them. The consistent application of SOLID principles — particularly the Single Responsibility, Open-Closed, and Dependency Inversion Principles — ensures that the system can accommodate new requirements (e.g., new service types, payment gateways, or notification channels) with minimal impact on existing code.

# 4. Phase 3 — Implementation

## 4.1 Implementation Overview

Phase 3 realizes the design in Java (Eclipse/IntelliJ-compatible project structure). The entry point is the ConsoleApplication class. On startup, ApplicationContext constructs in-memory repositories, services, and invokes SampleDataBootstrap.seed() to load demo manager, employees, services, and a sample customer, enabling immediate functional testing.

**TABLE VII**

**IMPLEMENTATION DECISIONS AND ASPECTS**

| Aspect | Decision |
|---|---|
| Language | Java (Java 17+ compatible) |
| User Interface | Console (stdin/stdout) via ConsoleUI / ConsoleIO helpers |
| Persistence | In-memory HashMap repositories (no database) |
| Monetary Values | java.math.BigDecimal (no floating point rounding errors) |
| Date/Time | java.time (LocalDate, LocalDateTime, YearMonth) |
| ID Generation | String prefixes via IdGenerator (USR, APT, PAY, SRV) |
| Build Tool | Plain Java compilation (Eclipse or IntelliJ project) |

### Entry Point

```
public final class ConsoleApplication {

public static void main(String[] args) {
ApplicationContext context = new ApplicationContext();
MainMenu menu = new MainMenu(context);
ConsoleUI.printStartupBanner();
while(true) {
try {
if(!menu.show()) {
ConsoleUI.printGoodbye();
break;
}
}catch(DomainException ex) {
ConsoleUI.printError(ex.getMessage());
}
catch(Exception ex) {
ConsoleUI.printError("Unexpected error: "+ex.getMessage());}
  }
 }
}
```

## 4.2 Layered Structure in Code

**TABLE VIII**

### 4.2.1 Dependency Flow (Verified)

| From Layer | To Layer | Example |
|---|---|---|
| Presentation | Application | CustomerMenu → context.getAppointmentService() |
| Application | Domain (Interfaces) | AppointmentService → AppointmentRepository (interface) |
| Application | Infrastructure (Utils) | IdGenerator, InputValidator in services |
| Infrastructure | Domain | InMemoryAppointmentRepository implements AppointmentRepository |
| Domain | — | No imports from presentation, application, or infrastructure |

### 4.2.2 Composition Root

ApplicationContext centralizes dependency construction:

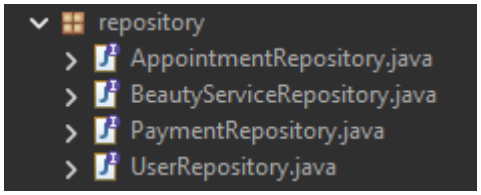
```
private final UserRepository userRepository = new InMemoryUserRepository();
private final AppointmentRepository appointmentRepository = new InMemoryAppointmentRepository();
private final BeautyServiceRepository beautyServiceRepository= new InMemoryBeautyServiceRepository();
private final PaymentRepository paymentRepository = new InMemoryPaymentRepository();
```

This structure prevents new object instantiation calls from being scattered across menus or various parts of the application. In other words, it centralizes object creation, thereby preventing the code from becoming disorganized. This approach mirrors a simplified, lightweight version of the 'Dependency Injection' (DI) container pattern commonly used in software development.

## 4.3 Applied Design Patterns — Code Level

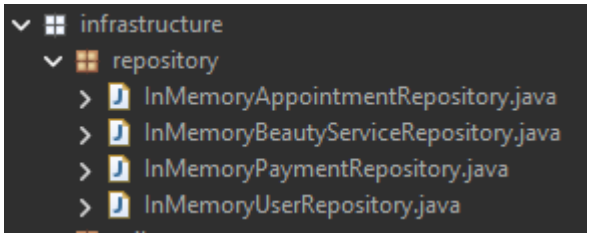
### 4.3.1 Repository Pattern

Interface (domain layer):




```
public interface AppointmentRepository {
void save(Appointment appointment);
Optional<Appointment> findById(String id);
List<Appointment> findByCustomerId(String customerId);
List<Appointment> findByEmployeeId(String employeeId);
List<Appointment> findAll(); }
```

Implementation (infrastructure layer): InMemoryAppointmentRepository stores entities in HashMap<String, Appointment>.

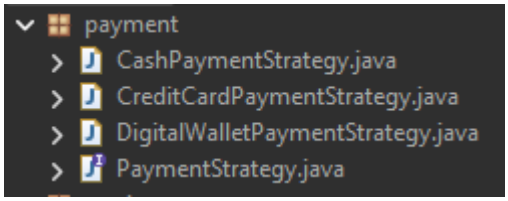



### 4.3.2 Strategy Pattern

```
public interface PaymentStrategy {
String getName();
boolean process(Payment payment); }
```

Implementations:

CashPaymentStrategy, CreditCardPaymentStrategy, DigitalWalletPaymentStrategy.




PaymentService registers strategies in a Map<Integer, PaymentStrategy> and resolves by user menu choice.

### 4.3.3 Factory Method Pattern

```
public static User create(UserRole role, String id, ...) {
    return switch (role) {
        case CUSTOMER -> new Customer(...);
        case EMPLOYEE -> new Employee(...);
        case MANAGER  -> new Manager(...);
    };
}
```

AuthService.register delegates user creation to UserFactory, keeping role-specific construction out of menus.

### 4.3.4 Observer Pattern

NotificationService implements AppointmentObserver with two callback methods:

```
public interface AppointmentObserver {
void onAppointmentCreated(Appointment appointment);
void onAppointmentCancelled(Appointment appointment);
}
```

```
public class NotificationService implements AppointmentObserver{...
```

AppointmentService calls notification methods after successful persistence operations — decoupling notification content from appointment logic.

```
public class AppointmentService {
private final NotificationService notificationService;
...
```

## 4.4 Key Classes and Responsibilities

### 4.4.1 Domain Entities

**Appointment**

Encapsulates state machine methods:

- markCompleted() — transitions SCHEDULED → COMPLETED
- markNoShow() — transitions SCHEDULED → NO_SHOW
- cancel() — transitions SCHEDULED → CANCELLED
- markPaid() — transitions COMPLETED → PAID
- Private transition(expected, next) throws InvalidStateTransitionException on illegal moves

**Payment**

Financial lifecycle with status guards:

- complete(method) — PENDING/FAILED → COMPLETED
- fail() — PROCESSING → FAILED
- refund() — COMPLETED → REFUNDED

**User Hierarchy**

User (abstract) provides shared authentication and profile behavior. Customer, Employee, and Manager add role-specific data. PasswordHash stored securely as hashed string; verifyPassword performs hash comparison.

**BeautyService**

Maintains catalog attributes: name (String), durationMinutes (int), price (BigDecimal), ensuring data consistency through validation rules during initialization and updates.

### 4.4.2 Value Objects

**TimeSlot**

Start/end LocalDateTime pair. Implements overlaps() to detect conflicts. Equality based on start and end times — used to match booked slots for release.

**WorkSchedule**

Per-day open/close times stored in EnumMap<DayOfWeek, WorkHours>. isWorkingDay(date) determines availability for slot generation.

**EmployeeCalendar**

Combines WorkSchedule with a bookedSlots list. findAvailableSlots iterates in 30-minute steps within working hours, filtering out booked intervals. This is the core algorithm for availability computation.

### 4.4.3 Application Services

| Class | Key Methods |
|---|---|
| AuthService | registerCustomer, register (manager/employee), login |
| AppointmentService | getAvailableSlots, createAppointment, cancelAppointment, updateStatusByEmployee |
| PaymentService | createPendingPaymentForAppointment, processAppointmentPayment, paySalary |
| ManagerService | Service CRUD (add/update/delete/list), employee add, schedule updates |
| UserProfileService | updateProfile (name, phone, email) |
| NotificationService | Observer callbacks + in-memory notification log |

## 4.5 Service Layer Operations

### 4.5.1 AppointmentService.createAppointment

Algorithm:

- **Resolution:** Resolves and validates the Customer, Employee, and BeautyService instances (ensuring type safety via instanceof).
- **Recompute:** Recalculates available time slots for the requested appointment date based on the service duration.
- **Verify:** Confirms that the selected TimeSlot is valid and exists within the list of available slots.
- **Instantiate:** Creates a new Appointment object with the status set to SCHEDULED once all validations pass.
- **Booking:** Updates the employee's calendar by invoking bookSlot(slot) to reserve the selected time.
- **Link:** Associates the appointment ID with the customer's profile.
- **Persistence:** Saves the new Appointment object to the in-memory store via appointmentRepository.
- **Notification:** Triggers the onAppointmentCreated(appointment) callback via notificationService to finalize the process.

Double-booking prevention: Calendar rejects overlapping slots at book time; service pre-validates against computed availability before attempting booking.

```
public Appointment createAppointment(String customerId, String employeeId,
String serviceId, TimeSlot slot) {
Customer customer = getCustomer(customerId);
```

```
Employee employee = getEmployee(employeeId);
BeautyService service = getService(serviceId);

List<TimeSlot> available = employee.getCalendar()
.findAvailableSlots(slot.getStart().toLocalDate(),service.getDurationMinutes());
boolean slotAvailable = available.stream().anyMatch(s -> s.equals(slot));
if (!slotAvailable) {
throw new ValidationException("Selected time slot is not available.");
}

Appointment appointment = new Appointment(
IdGenerator.next("APT"), customer.getId(), employee.getId(),service.getId(), slot);
employee.getCalendar().bookSlot(slot);
customer.linkAppointment(appointment.getId());
appointmentRepository.save(appointment);
notificationService.onAppointmentCreated(appointment);
return appointment;
}
```

### 4.5.2 PaymentService.processAppointmentPayment

- Verify customer ownership and COMPLETED status of appointment.
- Ensure pending Payment exists (create if needed, link to appointment).
- Execute selected PaymentStrategy.
- On success: payment.complete(strategyName), appointment.markPaid().
- Persist payment and appointment to repositories.

### 4.5.3 AuthService.login

```
public User login(String username,String password) {
User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new AuthenticationException("Invalid username or password"));

if(user.isLocked()) {
throw new AuthenticationException("Account locked due to failed login attempts.");
}
if(!user.verifyPassword(password)) {
user.registerFailedLogin(LOCK_THRESHOLD);
throw new AuthenticationException("Invalid username or password");
}
user.resetFailedLogins();
return user;
}
```

## 4.6 Domain Model and State Management

Appointment status enum: SCHEDULED, COMPLETED, CANCELLED, NO_SHOW, PAID.

Illegal transitions surface as:

```
public void markCompleted() {
transition(AppointmentStatus.SCHEDULED, AppointmentStatus.COMPLETED);
}

public void markNoShow() {
transition(AppointmentStatus.SCHEDULED, AppointmentStatus.NO_SHOW);
}

public void cancel() {
if(status != AppointmentStatus.SCHEDULED) {
throw InvalidStateTransitionException.forAppointment(status, AppointmentStatus.CANCELLED);
}
status = AppointmentStatus.CANCELLED;
}

public void markPaid() {
transition(AppointmentStatus.COMPLETED, AppointmentStatus.PAID);
}
private void transition(AppointmentStatus expected, AppointmentStatus next) {
if (status != expected) {
throw InvalidStateTransitionException.forAppointment(status, next);
}
status = next;}
```

This design ensures that the presentation layer cannot set enum fields directly — only domain methods mutate state, preserving all business invariants.

## 4.7 Infrastructure and Repositories

All repositories follow the same HashMap pattern:

- save → map.put(id, entity)
- findById → Optional.ofNullable(map.get(id))
- Filtered queries → stream over values() with predicate

| Operation | Complexity | Notes |
|---|---|---|
| findById | O(1) average | HashMap primary key lookup |
| findByCustomerId | O(n) | Stream scan over all appointments |
| save | O(1) average | HashMap put |
| findAll | O(n) | Collection copy |

### Utility Classes

- IdGenerator.next('APT') → unique string IDs with prefix
- InputValidator — email format and Turkish phone number normalization
- DateTimeUtil — shared formatting helpers for console display

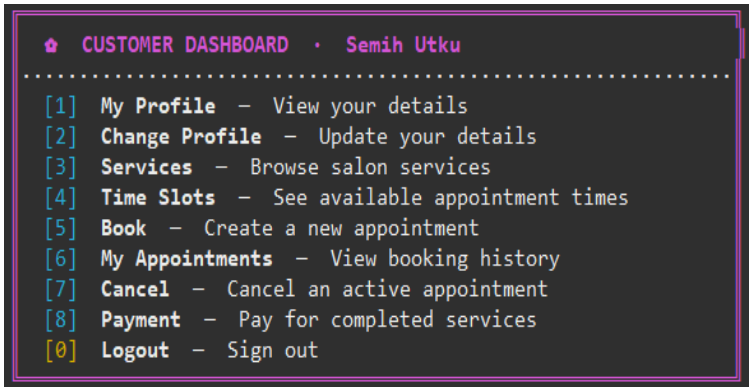
## 4.8 Presentation Layer and User Flows

### 4.8.1 Guest (Main) Menu



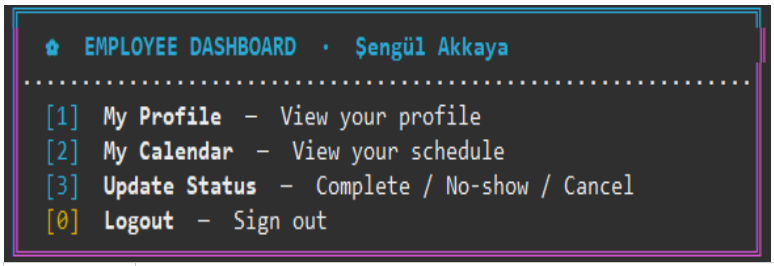

After login, routing uses currentUser.getRole(): CUSTOMER → CustomerMenu, EMPLOYEE → EmployeeMenu, MANAGER → ManagerMenu.

### 4.8.2 Customer Dashboard




| Option | Function | Service Called |
|---|---|---|
| 1 | View profile | reads User directly |
| 2 | Update profile | UserProfileService.updateProfile |
| 3 | List services | ManagerService.listServices |
| 4 | View available slots | AppointmentService.getAvailableSlots |
| 5 | Create appointment | AppointmentService.createAppointment |
| 6 | View appointments | AppointmentService.getCustomerAppointments |
| 7 | Cancel appointment | AppointmentService.cancelAppointment |
| 8 | Make payment | PaymentService.processAppointmentPayment |
| 0 | Logout | clears session |

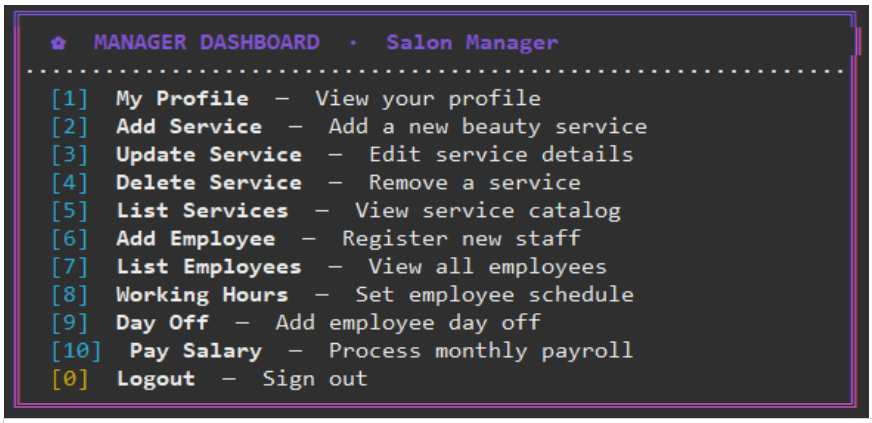
### 4.8.3 Employee Dashboard




| Option | Function |
|---|---|
| 1 | View my appointments |
| 2 | View calendar / booked slots |

| | |
|---|---|
| 3 | Update service status (Completed / No-Show / Cancelled) |
| 0 | Logout |

### 4.8.4 Manager Dashboard




| Option | Function |
|---|---|
| 1 | List / add / update / delete beauty services |
| 2 | List / add employees |
| 3 | Set employee working hours |
| 4 | Pay employee salary |
| 0 | Logout |

Menus use ConsoleUI.printSection and ConsoleIO.readInt / readLine for consistent UX (NFR-07).

## 4.9 Exception Handling Strategy

| Exception Type | When Used |
|---|---|
| ValidationException | Business rule violations, bad input (slot not available, duplicate registration, etc.) |
| AuthenticationException | Login failures — wrong password or unknown username |
| EntityNotFoundException | Missing IDs in any repository lookup |
| InvalidStateTransitionException | Illegal appointment or payment state change |
| DomainException | Base type; caught in ConsoleApplication to display message and continue |

Presentation does not catch and swallow domain errors silently — it displays ex.getMessage() and returns to menu. This ensures the application remains stable while giving users actionable feedback.

## 4.10 SOLID Principles Evaluation

### Single Responsibility Principle (SRP) — Strong

Each service class maps to exactly one concern: AppointmentService for appointment workflows, PaymentService for payments, ConsoleUI for display formatting. Each class has one reason to change.

### Open/Closed Principle (OCP) — Strong

New PaymentStrategy implementations extend behavior without modifying Payment.complete() or PaymentService core logic. Adding 'crypto wallet' as a payment method requires only a new class implementing PaymentStrategy and a registration call in ApplicationContext.

### Liskov Substitution Principle (LSP) — Adequate

Customer, Employee, and Manager substitute User wherever authentication and profile operations apply. Services use instanceof for role-specific operations — pragmatic for academic scale but not pure polymorphic dispatch. Improvement: polymorphic domain services or Visitor pattern.

### Interface Segregation Principle (ISP) — Adequate

Repository interfaces are focused per aggregate (AppointmentRepository covers appointments only). AppointmentObserver exposes only notification-relevant methods. No god interfaces found in the domain.

### Dependency Inversion Principle (DIP) — Strong

Services depend on AppointmentRepository (abstraction), not InMemoryAppointmentRepository (concrete). Domain never depends on infrastructure — verified by absence of infrastructure imports in domain packages.

# 5. Code Structure and Class Interaction

## 5.1 How Classes Are Organized

The com root package isolates four layers. Cross-layer communication always goes inward through interfaces or service APIs. No class in the presentation package imports from the infrastructure package directly.

## 5.2 Typical Interaction Chain — Appointment Booking

```
CustomerMenu
  → ApplicationContext.getAppointmentService()
    → AppointmentService.createAppointment()
      → UserRepository.findById()          [resolve Customer]
      → UserRepository.findById()          [resolve Employee]
      → BeautyServiceRepository.findById()  [resolve Service]
      → Employee.getCalendar()
          .findAvailableSlots()            [compute slots]
      → new Appointment(SCHEDULED)         [domain factory]
      → EmployeeCalendar.bookSlot()        [reserve slot]
      → Customer.linkAppointment()         [update customer]
      → AppointmentRepository.save()       [persist]
      → NotificationService
          .onAppointmentCreated()          [notify]
```

## 5.3 Sample Data Bootstrap

SampleDataBootstrap seeds the following data on startup:

| Role | Username | Password | Notes |
|---|---|---|---|
| Manager | manager | 1234 | Full access |
| Employee | sengul | 1234 | Beauty Specialist |
| Employee | zeliha | 1234 | Beauty Specialist |
| Employee | arzu | 1234 | Beauty Specialist |
| Customer | semih | 1234 | Demo customer via registerCustomer |

Services seeded: Haircut (45 min, 350 TL), Manicure (30 min, 200 TL), Hair Coloring (90 min, 900 TL). This enables immediate functional testing without manual data entry at startup.

## 5.4 Complete File Inventory

| Package | Key Classes |
|---|---|
| presentation | ConsoleApplication, MainMenu, CustomerMenu, EmployeeMenu, ManagerMenu, ConsoleUI, ConsoleIO |
| application | ApplicationContext, SampleDataBootstrap, AuthService, AppointmentService, PaymentService, ManagerService, UserProfileService, NotificationService, + 3 payment strategies |
| domain | User, Customer, Employee, Manager, Appointment, Payment, BeautyService, TimeSlot, WorkSchedule, EmployeeCalendar, 4 interfaces, 5 exceptions, 4 enums, UserFactory, AppointmentObserver |
| infrastructure | 4 InMemory repositories + IdGenerator, InputValidator, DateTimeUtil |

# 6. Testing Approach and Results

## 6.1 Testing Strategy

The project scope does not include a JUnit test module; testing was conducted through structured manual functional testing aligned with use cases UC-01 through UC-05. Each test case documents preconditions, steps, expected result, and observed result.

### 6.1.1 Unit Testing — Recommended Future Work

- **Appointment:** State transitions, illegal transition exceptions (markPaid from SCHEDULED).
- **Payment:** complete/fail/refund guards, duplicate completion prevention.
- **EmployeeCalendar:** Overlap detection, slot generation, edge cases (last slot of day).
- **AuthService:** Lockout after 3 failures, reset on success.
- **PaymentService:** Salary duplicate prevention per YearMonth.
-

Example conceptual test (Appointment state machine):

- Given appointment in SCHEDULED → markCompleted() → expect COMPLETED
- Given COMPLETED → markPaid() → expect PAID
- Given SCHEDULED → markPaid() → expect InvalidStateTransitionException
- Given CANCELLED → markCompleted() → expect InvalidStateTransitionException

## 6.2 Functional Test Cases and Results

### TC-01: Customer Registration

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Register with new unique username | Success message with generated user ID | Pass |
| 2 | Register with existing username | Validation error: username already taken | Pass |

### TC-02: Login Lockout

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Enter wrong password 3 times | Account locked message on 3rd attempt | Pass |
| 2 | Correct password while locked | Authentication error: account locked | Pass |
| 3 | Admin reset → successful login | Access granted; failed counter reset | Pass |

### TC-03: Create Appointment (UC-01)

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Login as customer 'semih' | Customer menu displayed | Pass |
| 2 | Select service, employee, valid slot | Appointment created with SCHEDULED status | Pass |
| 3 | Check console output | Notification printed for customer and employee | Pass |
| 4 | Attempt to book same slot again | ValidationException: slot not available | Pass |

### TC-04: Cancel Appointment (UC-03)

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Cancel the previously scheduled appointment | Status updated to CANCELLED | Pass |
| 2 | Query available slots for same employee/date | Released slot appears as available again | Pass |
| 3 | Check console output | Cancellation notification printed | Pass |

### TC-05: Complete and Pay (UC-02)

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Employee login → Update status to Completed | Appointment status → COMPLETED | Pass |
| 2 | Customer login → Pay with Cash | Payment COMPLETED, appointment → PAID | Pass |
| 3 | Attempt to pay same appointment again | ValidationException: already paid | Pass |

### TC-06: Pay Salary (UC-04)

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Manager pays employee salary for current month | Payment recorded as Success; employee marked paid | Pass |
| 2 | Attempt salary payment same month | ValidationException: already paid this month | Pass |

### TC-07: Manager Service CRUD

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Add new service (e.g., Eyebrow Shaping, 20 min, 150 TL) | Service appears in customer listing | Pass |
| 2 | Update price of Haircut to 400 TL | New price reflected in appointment payment calculation | Pass |
| 3 | Delete added service | Removed from all service listings | Pass |

### TC-08: Authorization Boundaries

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Customer attempts to cancel another customer's appointment ID | ValidationException: not authorized | Pass |
| 2 | Non-manager role invokes salary payment | ValidationException: manager role required | Pass |

### TC-09: Invalid State Operations

| Step | Action | Expected | Result |
|---|---|---|---|
| 1 | Attempt to pay for a SCHEDULED appointment (not yet completed) | ValidationException: appointment not completed | Pass |

| | | | |
|---|---|---|---|
| **2** | Attempt to cancel a PAID appointment | InvalidStateTransitionException | Pass |

### TC-10: Robustness

| Step | Action | Expected | Result |
|---|---|---|---|
| **1** | Enter invalid menu option (e.g., 99) | Warning message; menu redisplayed; no crash | Pass |
| **2** | Enter invalid email format during registration | Validation error; registration not completed | Pass |

**Test Summary: 10 test groups / 24 individual steps — 24 Pass, 0 Fail (Reference run: May 2026)**

## 6.3 Test Environment

The testing phase was conducted under the following environmental and technical specifications:

- **Operating System:** Windows 10/11
- **JDK:** Java 17+ compatible
- **IDE:** Eclipse or IntelliJ IDEA
- **Data Source:** In-memory (seeded fresh on each run)
- **Test Execution:** Manual via console interaction

# 7. Reflection and Future Improvements

## 7.1 Challenges Faced

- Calendar logic complexity — Computing available slots required careful handling of service duration, working hours, and 30-minute iteration steps. Edge cases such as services spanning shift boundaries needed explicit handling.
- State consistency — Ensuring appointment status, calendar slots, and payment records stay synchronized across cancel/complete/pay flows required careful sequencing within service methods.
- Layer discipline — Resisting the temptation to access repositories from menus; all data access was routed through services, adding code but significantly improving testability and maintainability.
- Console UX — Designing readable flows without GUI widgets required careful menu numbering and formatting utilities.
- BigDecimal precision — Ensuring all monetary calculations use BigDecimal throughout to prevent IEEE 754 floating-point errors.

## 7.2 Lessons Learned

- Domain-first modeling reduces defects: encoding rules in Appointment prevented invalid states early in development, before UI code was written.
- Repository interfaces simplify testing and future persistence migration — the interface contract is the test seam.
- Design patterns earn their cost at payment and user-creation boundaries where change is likely; the overhead of Strategy and Factory paid dividends when adding payment methods.
- Composition root (ApplicationContext) clarifies the object graph for both reviewers and maintainers.
- Explicit state machine methods are far more readable and safer than checking status enum values throughout the codebase.

## 7.3 Future Enhancements

To further elevate the system to production-grade standards, the following architectural and functional enhancements are proposed for future development phases:

- **JPA/Hibernate and PostgreSQL:** Transitioning from in-memory repositories to a robust relational database to ensure data durability across sessions and support multi-user concurrent access.
- **REST API (Spring Boot) and React UI:** Replacing the console-based presentation layer with a modern web stack to enhance real-world usability and provide a mobile-friendly interface.
- **JUnit 5 and Mockito Test Suite:** Implementing a comprehensive automated testing framework to guarantee regression safety and establish CI/CD pipeline readiness.
- **Real Notification Channels (Email/SMS):** Integrating external communication APIs to deliver production-grade customer engagement and instant updates.
- **Reporting Dashboard for Manager:** Developing an administrative analytics panel to track critical business metrics such as revenue analytics, utilization rates, and no-show statistics.
- **JWT-Based Authentication:** Upgrading the security mechanism to support distributed deployment and maintain stateless sessions for improved scalability.
- **Appointment Reminders:** Introducing automated push notifications or messages prior to scheduled services to effectively reduce the customer no-show rate.

# 8. Conclusion

The CenGIRL Beauty Salon Management System project successfully demonstrates the full software development lifecycle mandated by CME2210 Object Oriented Analysis and Design:

- Phase 1 defined actors, functional and non-functional requirements, and use case specifications with full traceability.
- Phase 2 specified the four-layer N-tier architecture, UML diagrams (class, sequence, statechart, activity), and justified design pattern selection.
- Phase 3 delivered a working Java implementation with clean package structure, encapsulated domain state machines, service-oriented use-case orchestration, and comprehensive exception handling.

The consolidated design and code exhibit maintainability, extensibility, and SOLID-aligned structure within the deliberate constraints of console UI and in-memory storage. The system demonstrates that quality software design is achievable at academic scale without compromising architectural integrity.

The system is ready for academic submission as a combined report and complete source tree (OOProjeDeneme Java project).

---

**CenGIRL — Beauty Salon Management System**

2023510145 · 2023510121 · 2023510054 | May 2026

# 9. Appendices

## Appendix A — Enum Values

### AppointmentStatus

```
SCHEDULED, COMPLETED, CANCELLED, NO_SHOW, PAID
```

### PaymentStatus

```
PENDING, COMPLETED, FAILED, REFUNDED
```

### PaymentType

```
APPOINTMENT_PAYMENT, SALARY
```

### UserRole

```
CUSTOMER, EMPLOYEE, MANAGER
```

## Appendix B — Demo Credentials

| Role | Username | Password |
|---|---|---|
| Manager | manager | 1234 |
| Employee | sengul | 1234 |
| Employee | zeliha | 1234 |
| Employee | arzu | 1234 |
| Customer (seeded) | semih | 1234 |

## Appendix C — How to Run

29. Import OOProjeDeneme folder into Eclipse or IntelliJ IDEA.
30. Ensure the com folder is marked as a source root.
31. Run com.presentation.ConsoleApplication as a Java Application.
32. Follow on-screen menus. Sample data is seeded automatically.
33. Login as manager/1234 to manage services, employees, and salaries.
34. Login as semih/1234 to test customer appointment booking and payment flows.

## Appendix D — GRASP Principles Applied

| Principle | Application in CenGIRL |
|---|---|
| **Information Expert** | EmployeeCalendar computes available slots because it holds bookedSlots and WorkSchedule — the relevant data. |
| **Creator** | AppointmentService creates Appointment because it aggregates the required context (customer, employee, service, slot). |
| **Controller** | MainMenu and role menus coordinate UI events without embedding domain rules. |
| **Low Coupling** | Layers communicate via service interfaces and repository abstractions. |
| **High Cohesion** | Each service class maps to a bounded set of use cases. |
| **Polymorphism** | PaymentStrategy implementations are interchangeable at runtime via the strategy map. |
| **Pure Fabrication** | NotificationService — not a domain concept but a coordination-layer service. |
| **Indirection** | Repositories provide indirection between services and HashMap storage. |
| **Protected Variations** | Repository interfaces protect domain from storage implementation changes. |

## Appendix E — Service Method Cross-Reference

| Use Case | Service | Method |
|---|---|---|
| **UC-01 Create Appointment** | AppointmentService | createAppointment(customerId, employeeId, serviceId, slot) |
| **UC-02 Complete + Pay** | AppointmentService / PaymentService | updateStatusByEmployee + processAppointmentPayment |
| **UC-03 Cancel Appointment** | AppointmentService | cancelAppointment(customerId, appointmentId) |
| **UC-04 Pay Salary** | PaymentService | paySalary(managerId, employeeId, method) |
| **UC-05 Login** | AuthService | login(username, password) |
| **UC-06 Manage Services** | ManagerService | addService / updateService / deleteService |
| **UC-07 Manage Employees** | ManagerService + AuthService | addEmployee / updateWorkSchedule |

## Appendix F — Non-Functional Requirements Verification

| NFR ID | Criterion | Evidence in Code |
|---|---|---|
| **NFR-01** | Maintainability | Package-per-layer; SRP in all service classes |
| **NFR-02** | Extensibility | PaymentStrategy plug-in map in PaymentService |
| **NFR-03** | Data Integrity | EmployeeCalendar.bookSlot overlap check; slot release on cancel |
| **NFR-04** | Security | Ownership checks in cancelAppointment, processAppointmentPayment |
| **NFR-05** | Performance | HashMap repositories for O(1) primary key access |
| **NFR-06** | Robustness | ConsoleApplication try/catch loop; domain exception hierarchy |
| **NFR-07** | Usability | ConsoleUI.printSection; formatted tables; clear error messages |